

Performance Tuning Workshop

Mehr Speed mit den richtigen Algorithmen und Datenstrukturen

WJAX München 2026 – Michael Inden

https://github.com/Michaeli71/WJAX-Performance-Tuning-Workshop-2026

Michael Inden



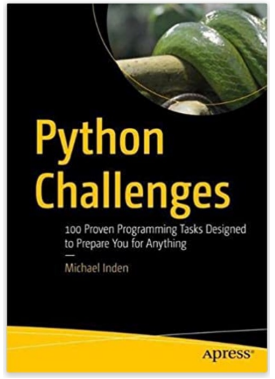
Speaker Intro Michael Inden

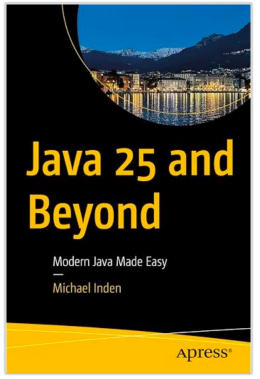


- Michael Inden, Jahrgang 1971, michael_inden@hotmail.com
- Diplom-Informatiker, C.v.O. Uni Oldenburg
- ~8 ¼ Jahre SSE bei Heidelberger Druckmaschinen AG in Kiel
- ~6 ¾ Jahre TPL, SA bei IVU Traffic Technologies AG in Aachen
- ~4 ¼ Jahre LSA / Trainer bei Zühlke Engineering AG in Zürich
- ~3 Jahre TL / CTO bei Direct Mail Informatics / ASMIQ in Zürich
- ~3 ½ Jahre Head of Development bei Adcubum in Zürich
- Ab 2026 Dozent Software Engineering und Studienleiter MAS SE
- Freiberuflicher Trainer und Konferenz-Speaker
- Autor bei dpunkt.verlag, O'Reilly und APress

















Agenda heute

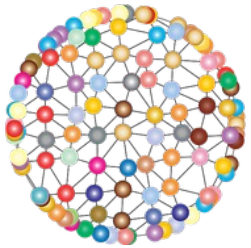
1. 🎬 Einstieg
2. 🧠 Performance verstehen: Big O und Skalierung
3. 🗃️ Wichtige Datenstrukturen und ihre Eigenschaften
4. 🔎 Suchen und Sortieren
5. 💡 Cleverere Lösungen: Two Pointers und HashMaps
6. Σ Präfixsummen
7. 🪟 Sliding Windows
8. 🧩 Rekursion und Memoization
9. 🌳 Tries und Autocomplete
10. 🏁 Performance-Finale: messen, erklären, optimieren
11. 🔥 Performance Fireworks
12. 🎯 Fazit





Think: Good code. Bad performance?





Wir alle kennen diese Situationen …

- Unser Code sieht gut aus, ist elegant, gut lesbar und sogar verständlich ☺
- Aber leider hat sich irgendwo eine Performance-Bremse eingeschlichen. Wieso ist alles so unerwartet langsam? ☹

Was würdet ihr tun, wenn das Programm bei kleinen Datenmengen schnell ist, bei großen aber plötzlich viel langsamer wird?



🎬 Einstieg
„Warum ist das langsam?“





Motivation: Datastrukturen sind relevant

- Dieser **Alltagscode** wirkt harmlos, korrekt und praktisch:

```
static int countDesiredNumbers(Collection<Integer> idsToValidate,
                               Collection<Integer> knownIds) {
    int count = 0;
    for (Integer id : idsToValidate) {
        if (knownIds.contains(id)) {
            // process(id);
            count++;
        }
    }
    return count;
}
```

- **Ist dieser Code schnell oder langsam?**
  - Bei 1.000 IDs? Bei 100.000 IDs? Bei 10 Millionen IDs?



Takeaway
Im Zweifel immer (gründlich) messen (idealerweise mehrfach).



DEMO: `Motivation.java` mit und ohne Warm-Up





Key Takeaways

- Beide Versionen arbeiten **korrekt!**
- Beide sind **lesbar!**
- Beide wirken **harmlos!**
- **Aber ihr Laufzeitverhalten ist DRAMATISCH unterschiedlich:**
- Das Problem ist **nicht Java's Syntax oder die APIs …**
- Das Problem liegt im **mentalen Modell und der Verwendung von (unpassenden) Datenstrukturen: `contains()` für Listen eher langsam, für Sets sehr schnell.**

Abhängig von Systemlast, typisch 500x – 1.500x, mit Warm-Up auch 8.000x

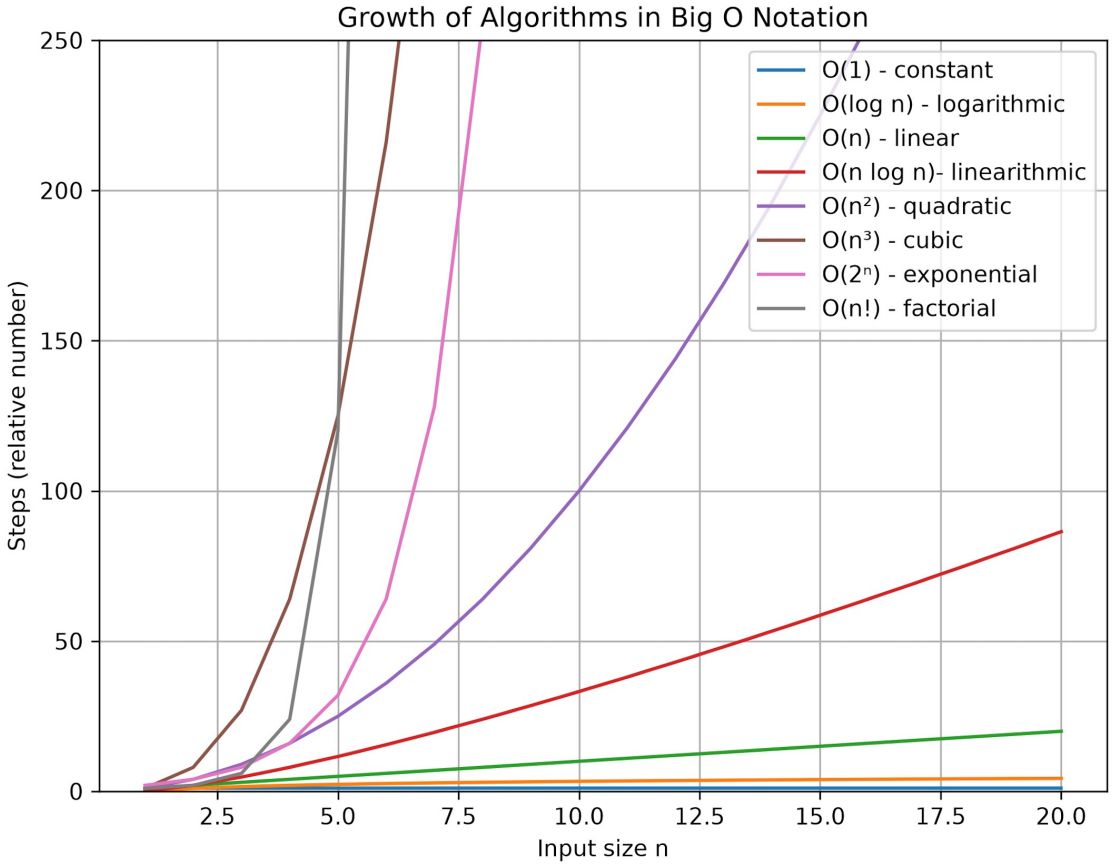
```
list: 1.7123s
set:  0.0022s
factor: 796.2x
```

Performance beginnt nicht mit der Optimierung von Code, sondern mit der Wahl der richtigen Algorithmen und Datenstrukturen.



🧠 Performance verstehen

Big O und Skalierung





Wachstum der Laufzeit mit zunehmender Eingabegröße

- **Laufzeitanalyse** = Zunahme der Ausführungszeit klassifizieren, um Effizienz bewerten zu können.

```
static boolean findValue(int[] values, int target) {
    for (int value : values) {
        if (value == target)
            return true;
    }
    return false;
}
```

„Was passiert, wenn der Datenbestand um Faktor 10 oder 100 größer wird?"

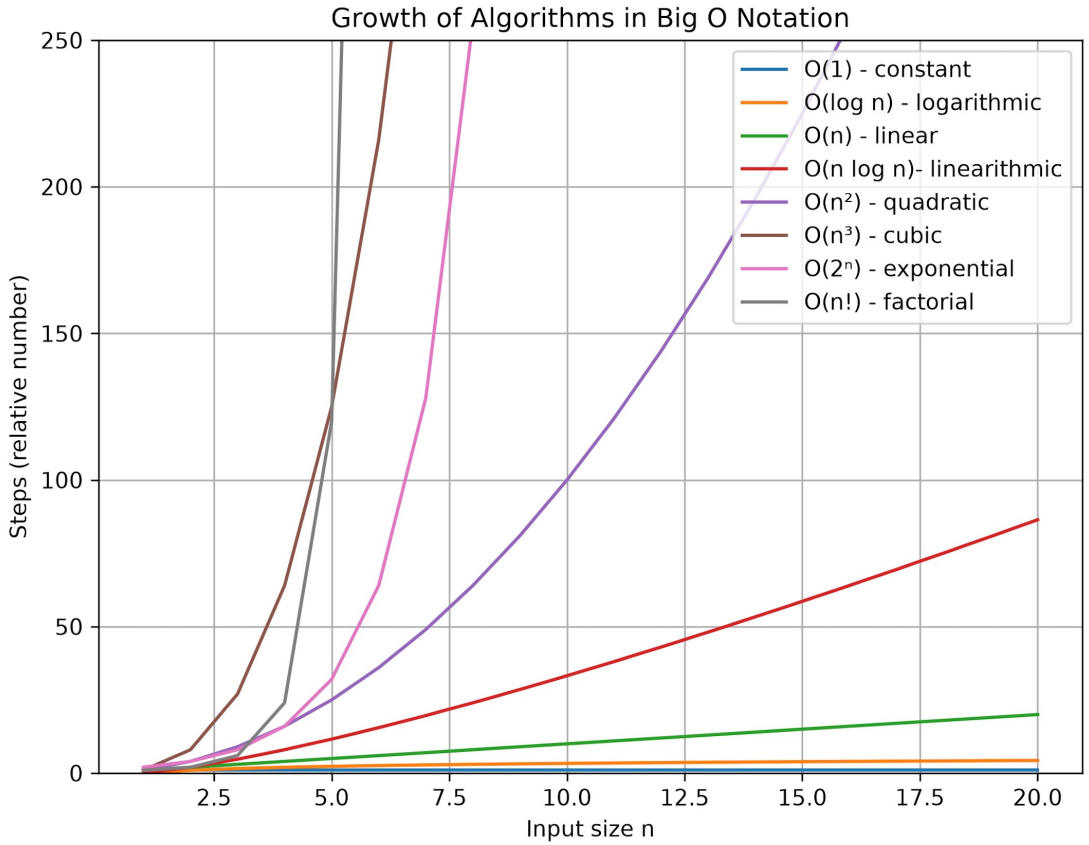
- **Lineare Skalierung**

  10'000 items → ~10'000 checks
  100'000 items → ~100'000 checks
  1'000'000 items → ~1'000'000 checks

Takeaway
Zur Beschreibung des Laufzeitwachstums verwenden wir eine spezielle Notation: **Big O**



Big O: Wachstum statt konkreter Laufzeit



Complexity	Name	Example
$O(1)$	constant	Accessing an array / list element by index
$O(\log n)$	logarithmic	Repeatedly halving the problem (e.g. “guess the number” with hints)
$O(n)$	linear	Iterating through a list once
$O(n \cdot \log n)$	linearithmic	Repeatedly dividing and combining
$O(n^2)$	quadratic	Two nested loops
$O(n^3)$	cubic	Three nested loops
$O(2^n)$	exponential	Trying all possibilities (e.g. determining all subsets)
$O(n!)$	factorial	Determining all permutations

„Wir werden einige Kurven bis zum Ende nahezu im flache Linien verwandeln.“

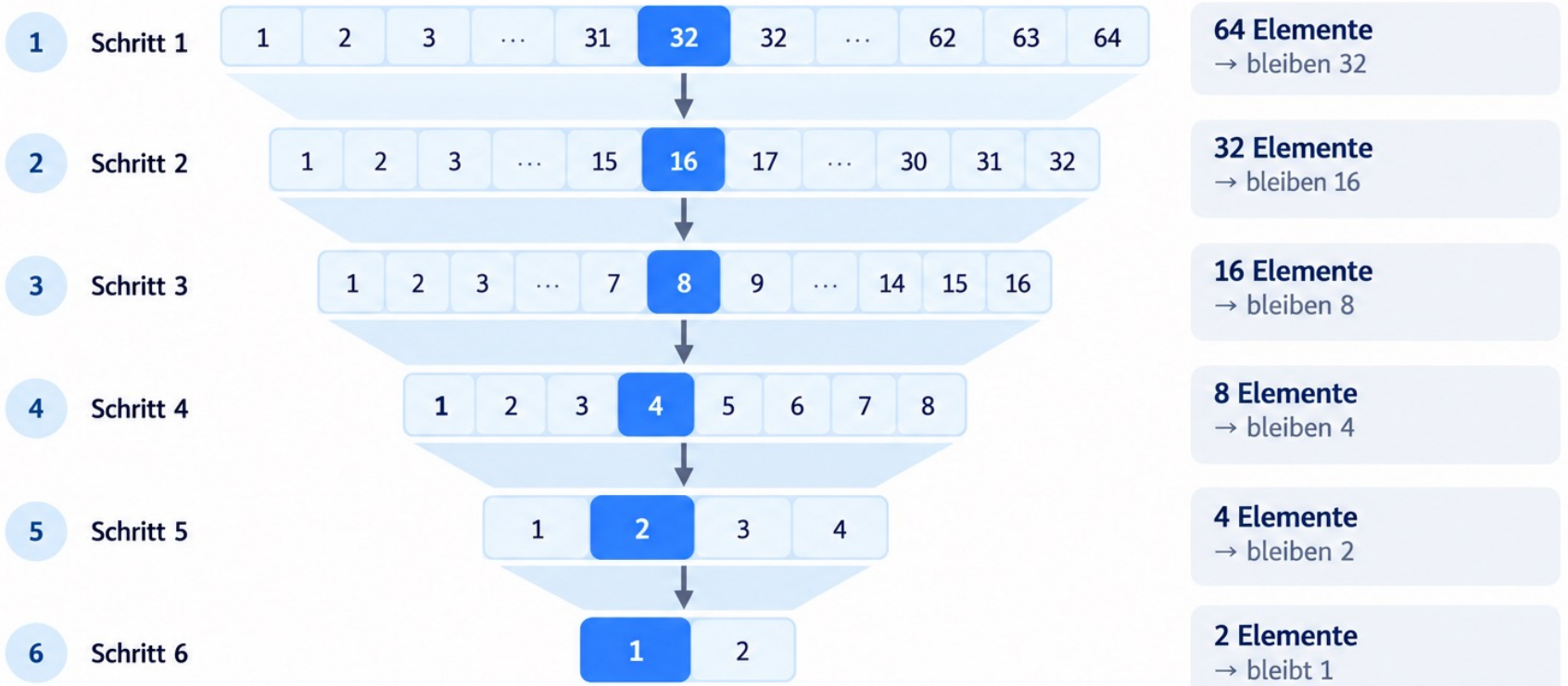


🧠 Performance verstehen

O-NOTATION

Logarithmisch – jedes Mal die Hälfte

In jedem Schritt wird die Menge der Elemente (z. B. ein Suchbereich) auf etwa die Hälfte reduziert.



$O(\log_2 n)$

Für **64 Elemente** sind höchstens **6 Schritte (Halbierungen)** nötig, denn $2^6 = 64$.

Merke:
Bei Verdopplung der Datenmenge kommt bei logarithmischem Wachstum ungefähr ein Schritt dazu oder der Aufwand halbiert sich!



Konstante Laufzeit – O(1)

Die Anzahl der Operationen bleibt gleich – unabhängig von der Größe der Eingabe.

DAS BEDEUTET

1 Schritt

Ob 10 oder 1.000.000 Werte vorhanden sind: Ein direkter Zugriff benötigt eine konstante Anzahl an Operationen.

BEISPIELE IN JAVA

```
static double divBy2(double n) {
    return n / 2;
}
```

```
static String pickSecond(String[] values) {
    if (values.length > 1) {
        return values[1];
    }
    return null;
}
```

Berechnung und indizierter Zugriff benötigen jeweils konstant viele Schritte.



Lineare Laufzeit – O(n)

Die Anzahl der Operationen wächst proportional zur Anzahl der Eingabewerte n.

1 · Alle Strings prüfen

```
int countEvenLength(String[] values) {
    int count = 0;
    for (String val : values) {
        if (val.length() % 2 == 0) {
            count++;
        }
    }
    return count;
}
```

2 · Längsten String finden

```
String findLongestString(String[] values) {
    String longest = "";
    for (String val : values) {
        if (val.length() > longest.length()) {
            longest = val;
        }
    }
    return longest;
}
```

1.000 Werte → ca. 1.000 Schleifendurchläufe

Konstante Zusatzschritte und Faktoren werden weggelassen: O(5n + 7) → O(n).



Quadratische Laufzeit – O(n²)

Verschachtelte Schleifen verarbeiten für jeden Wert erneut alle Werte.

BEISPIEL IN JAVA

```
List<String> allPairs(String[] values) {
    List<String> result = new ArrayList<>();
    for (String val : values) {
        for (String innerVal : values) {
            result.add("(" + val + ", " +
                       innerVal + ")");
        }
    }
    return result;
}
```

WIE SCHNELL WÄCHST DAS?

n = 10	10 × 10 = 100
n = 100	100 × 100 = 10.000
n = 1.000	1.000 × 1.000 = 1.000.000 Operationen

Beispiel „ABC“: 3 × 3 = 9 geordnete Paare. Allgemein gilt: $n \cdot n = n^2$.



Wie erkennt man typischerweise Komplexität im Code?

- Eine **Schleife** über alle Elemente: typischerweise O(n).
- **Zwei** ineinander **verschachtelte Schleifen** über dieselbe Eingabe: typischerweise $O(n^2)$.
- **Drei** ineinander **verschachtelte Schleifen** über dieselbe Eingabe: typischerweise $O(n^3)$.
- Dazu die wichtige Einschränkung: Entscheidend ist, wie oft die Operationen tatsächlich ausgeführt werden – nicht allein die Zahl der Schleifen im Quelltext.

Takeaway
Verschachtelte Schleifen möglichst vermeiden. Optimiere nicht nur den Code, sondern vor allem den benötigter Aktionen (oder den Algorithmus).



Optimierungsbeispiele Vorgriff auf später

Beispiel aus dem Workshop	Vorher → nachher	Was es verdeutlicht
Pair Sum mit Look-up-Map	$O(n^2) \rightarrow O(n)$	Zusätzliche Datenstruktur spart wiederholte Suche.
Fibonacci mit Memoization	$O(2^n) \rightarrow O(n)$	Identische Teilprobleme müssen nicht wiederholt berechnet werden.
Bereichsabfragen mit **Präfixsummen**	$O(n) \rightarrow O(1)^*$ $O(n^2) \rightarrow O(1)^*$ (für 2D)	Vorarbeit kann wiederholte Abfragen drastisch beschleunigen.

*pro Abfrage, aber vorab $O(n)/O(n^2)$ für Vorberechnung

Takeaway
Optimiere nicht nur Code – reduziere die zu leistende Arbeit.



Von Big O zur echten Laufzeit

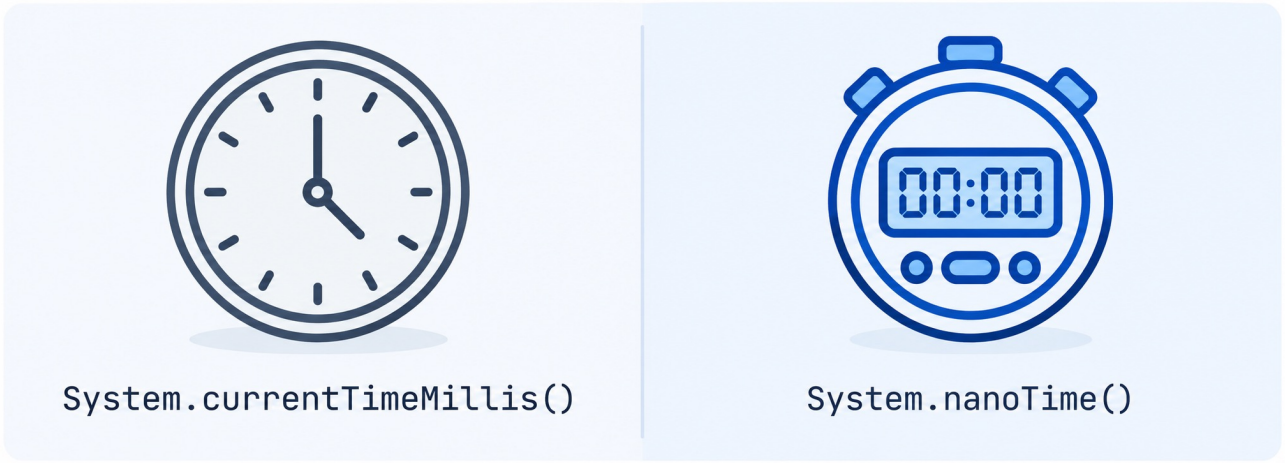
- Big O beschreibt, wie sich der Aufwand mit wachsender Eingabegröße entwickelt.
- **Damit kennt man jedoch noch nicht die tatsächliche Laufzeit!**
- Performance-Messungen zeigen, wie lange konkrete Läufe auf einem konkreten System dauern.
- **Beides ergänzt sich: Theorie erklärt das Wachstum, Performance-Messung macht es greifbar.**

Kernaussage: Wir messen nicht, um die Komplexität zu erraten
– wir nutzen die Komplexität, um Messungen einzuordnen.

- **Aber: Wie messen wir konkret …**



Laufzeitmessung: Stoppuhren in Java



- **`System.currentTimeMillis()`**
  - Millisekunden seit 1.1.1970
  - Gut für Zeitstempel und “grobe” Zeitmessungen
  - Für kurze Laufzeiten oft zu grob, weil ms
- **`System.nanoTime()`**
  - Für verstrichene Zeit gedacht
  - Normalerweise höhere Auflösung

Faustregel: Für Laufzeitdifferenzen `nanoTime()` bevorzugen – Nanosekunden-Einheiten bedeuten allerdings nicht automatisch auch Nanosekunden-Genauigkeit.



Laufzeitmessung: Zwei Lösungen, unterschiedliche Dauer

- **Duplikate in einer Liste erkennen (`hasDuplicates()`)**
  - **Variante A:** Jedes Element mit allen anderen vergleichen – verschachtelte Schleifen.
  - **Variante B:** Bereits gesehene Elemente in einem `HashSet` speichern.
- **Voraussetzungen und Vorgehen bei der Performance-Messung:**
  - Gleiche Eingabedaten und gleiche Aufgabe
  - Verschiedene Eingabegrößen
  - Laufzeiten mit `System.nanoTime()` messen
  - Messergebnisse kritisch hinterfragen

Wie verändert sich die Laufzeit mit wachsender Datenmenge / Eingabegröße – und passt das zu unserer Big-O-Analyse?



DEMO: `BigO_MotivationMeasurement.java`





Erkenntnis: Auf die Datenstruktur kommt es an

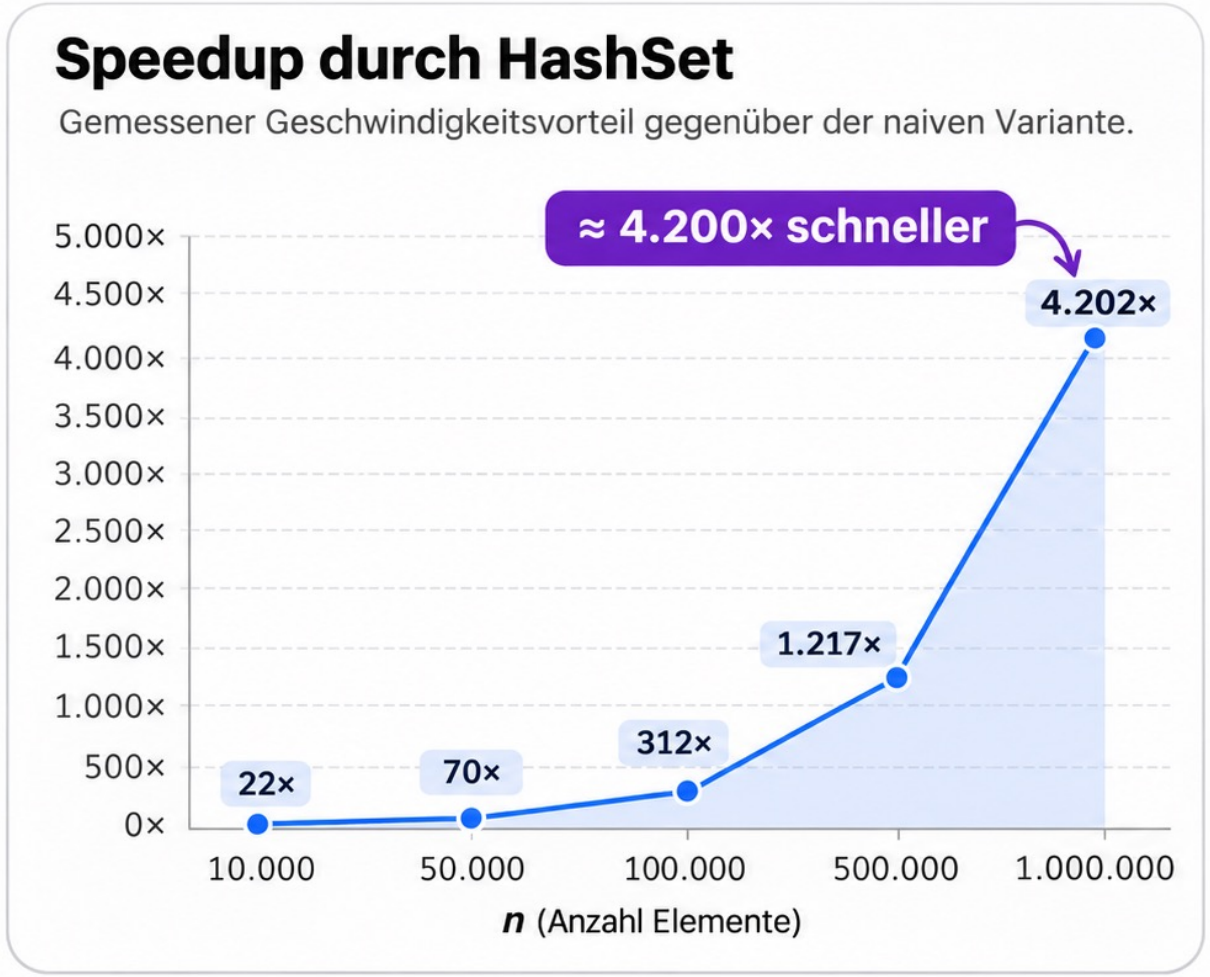
- **Das Programm liefert in etwa eine Tabelle mit Messwerten vergleichbar zu diesen:**

```
n=    1000 | Naiv:      1.364 ms | HashSet:      0.129 ms | Speedup:    10.57x
n=    5000 | Naiv:      1.528 ms | HashSet:      0.317 ms | Speedup:     4.82x
n=   10000 | Naiv:      9.952 ms | HashSet:      0.449 ms | Speedup:    22.17x
n=   50000 | Naiv:    141.483 ms | HashSet:      2.010 ms | Speedup:    70.40x
n=  100000 | Naiv:    539.808 ms | HashSet:      1.731 ms | Speedup:   311.88x
n=  500000 | Naiv:  13011.494 ms | HashSet:     10.691 ms | Speedup:  1217.09x
n= 1000000 | Naiv:  52266.267 ms | HashSet:     12.437 ms | Speedup:  4202.47x
```

- Die **Datenstruktur entscheidet**: `HashSet` statt verschachtelter Schleifen – von $O(n^2)$ zu $O(n)$.
- **Wachstum** macht den **Unterschied**: Je grösser die Datenmenge, desto stärker wirkt sich die bessere Komplexitätsklasse aus.
- **Zeit** gegen **Speicher**: Etwas zusätzlicher Speicher (O(n)) kann enorm viel Rechenzeit sparen.



Erkenntnis: Auf die Datenstruktur kommt es an



Auf einen Blick

	Laufzeit (Zeit)	Speicher (Memory)
Naiv Verschachtelte Schleifen	$O(n^2)$	$O(1)$
HashSet Mit Hash-Tabelle	$O(n)$	$O(n)$

Nicht nur den Code optimieren – die Menge der notwendigen Arbeit reduzieren!



Abgrenzung: Laufzeitmessung ist kein Benchmarking

- Zeitmessung wird möglicherweise beeinflusst durch:
  - JIT-Kompilierung und Warm-up
  - Garbage Collection und andere Prozesse
  - Betriebssystem und Hardware
  - Messaufwand und zu kurze Laufzeiten

💡 **Praxistipp: JIT & Warm-up**

Die JVM optimiert häufig ausgeführten Code zur Laufzeit durch den **Just-in-Time-Compiler (JIT)**.

- **Ohne Warm-up:** Erste Messungen können durch JIT-Effekte verfälscht sein.
- **Mit Warm-up:** Als Abhilfe, den Code vor der Messung mehrfach ausführen.

Merke: Einzelmessungen sind keine verlässlichen Benchmarks. Für belastbare Mikrobenchmarks: JMH.

Unser Vorgehen: Erst Algorithmus und Datenstrukturen verstehen und verbessern. Dann mit einem Profiler nach Engpässen suchen. Für belastbare Mikrobenchmarks später JMH einsetzen.



Exercises





🗂️ Wichtige Datenstrukturen und ihre Eigenschaften

`ArrayList` und `LinkedList (Queue)`

`HashSet / HashMap`





Think: Good code. Bad performance?



Wir müssen 1 Million IDs speichern …

```
List<String> ids = new ArrayList<>();
Set<String> ids2 = new HashSet<>();
```

Welche Variante würdet ihr wählen?



Grundlegende Fragestellungen

Suchen

Ist ein Element vorhanden?

Zugreifen

Wie schnell komme ich an Element i?

Einfügen und löschen

Wo ändere ich die Daten – und was muss verschoben werden?

Reihenfolge

Sind Indexzugriff, Sortierung oder geordnete Traversierung wichtig?

Kernaussage: Nicht „Welche Datenstruktur ist die beste?“, sondern „Welche Datenstruktur passt zu meinen häufigsten Operationen?“



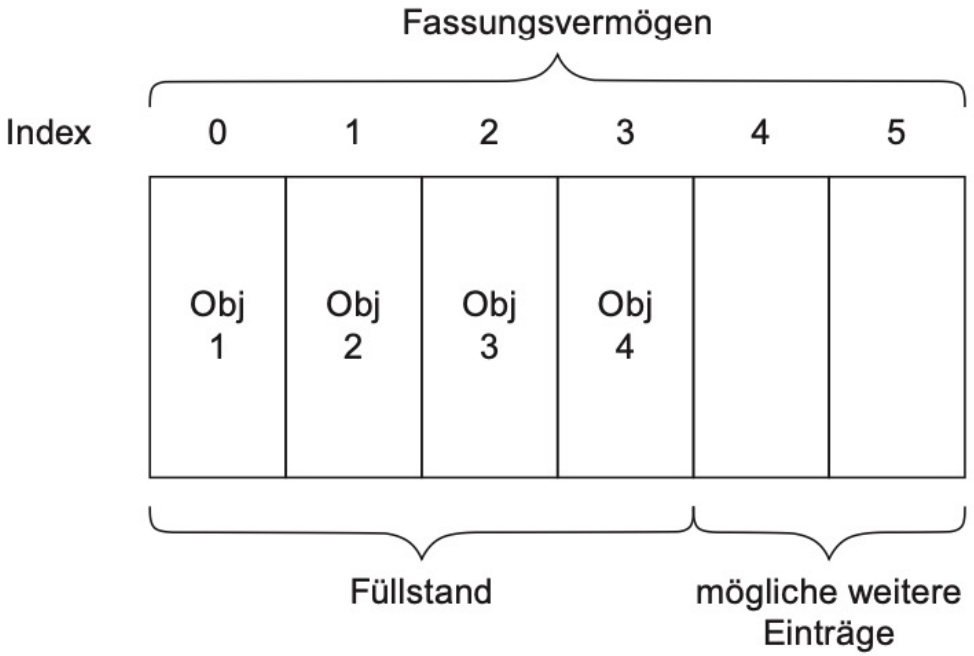
Verschiedene Datenstrukturen. Verschiedene Trade-offs!

Struktur	Stärke	Worauf achten?
Array / `ArrayList`	Indexzugriff $O(1)$	Einfügen oder Löschen in der Mitte kann Verschieben erfordern.
`LinkedList`	Änderungen an bekanntem Knoten	Indexzugriff und Suche benötigen typischerweise $O(\mathrm{n})$.
`HashSet`	Enthaltensein prüfen $O(1)$	Im Durchschnitt $O(1)$; keine verlässliche Iterationsreihenfolge.
`HashMap`	Schlüssel einem Wert zuordnen, Enthaltensein prüfen $O(1)$	Schlüssel müssen korrekt `equals()` und `hashCode()` implementieren.
`TreeSet` / `TreeMap`	Sortierte Schlüssel und geordnete Operationen	Typische Such- und Änderungsoperationen $O(\log \mathrm{n})$.

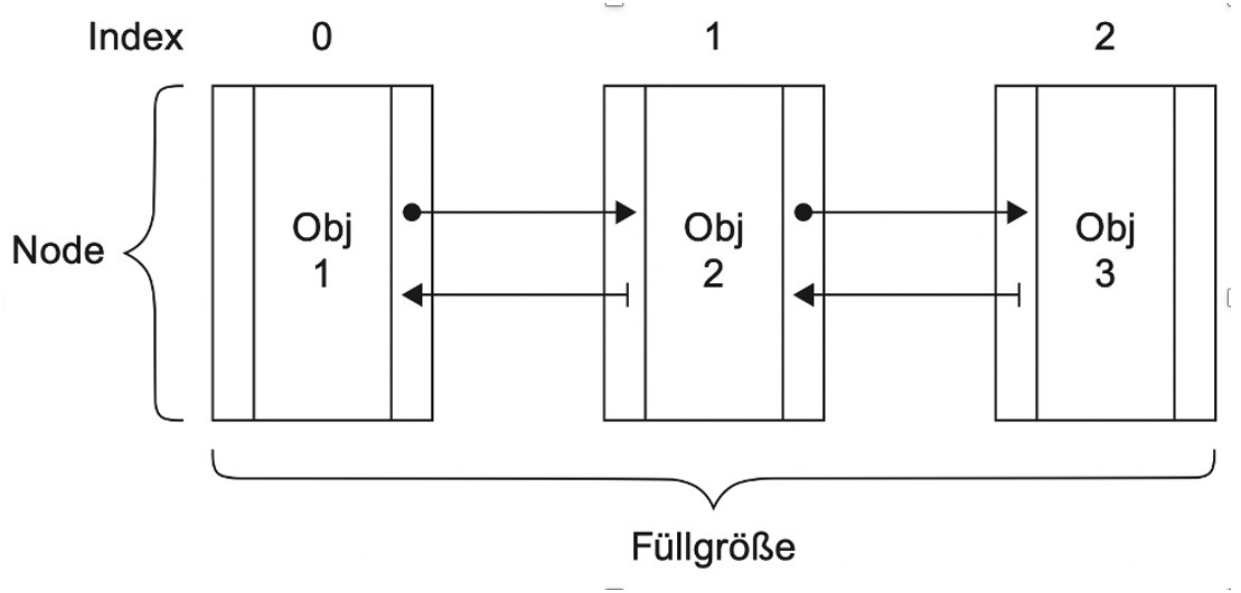


Grobaufbau von Listen

- `ArrayList`: Array-basiert



- `LinkedList`: Verknüpfte Knoten

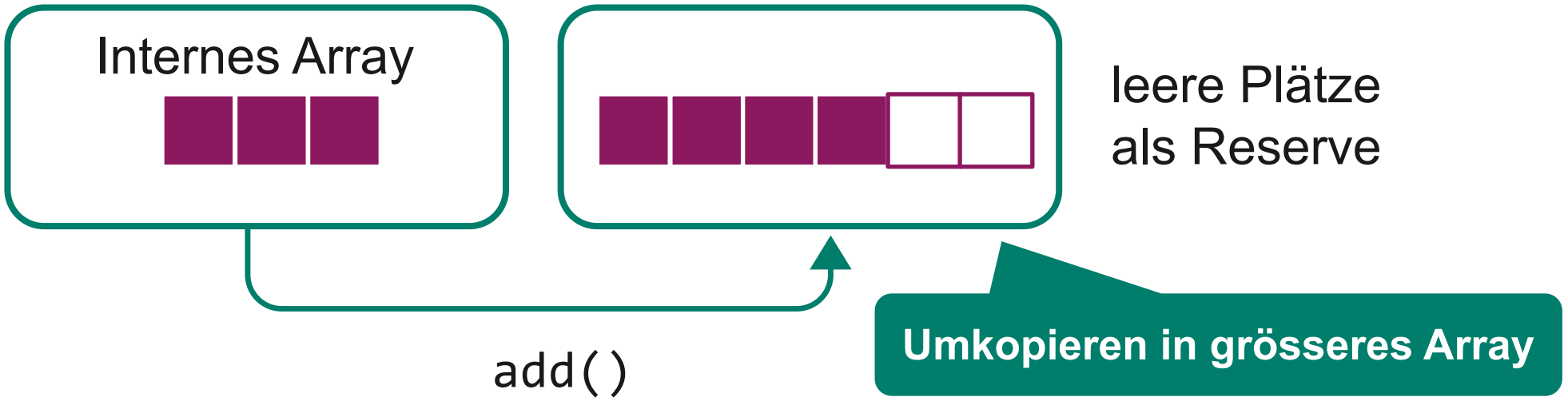


- Ordnung mit Index 0 bis `size() - 1`
- Duplikate und `null`-Einträge möglich



Besonderheit `ArrayList`

- Wie Array, aber veränderliche Anzahl an Elementen / Inhalt





Performance



Wie schätzen Sie die Performance von `ArrayList` ein?

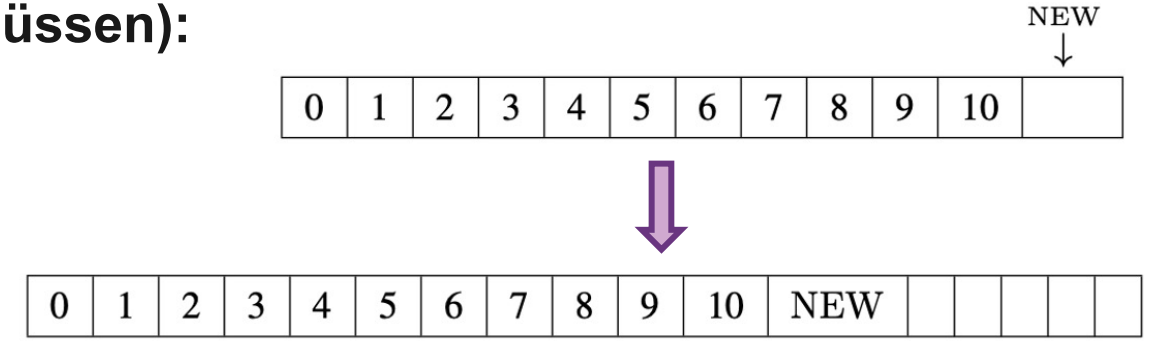
Operation	Methode	Effizienz
Index-Zugriff	`get(), set()`	
Einfügen eher am Ende	`add()`	
Entfernen vorne bis Mitte	`remove(int)`	
Suchen	`contains()`	



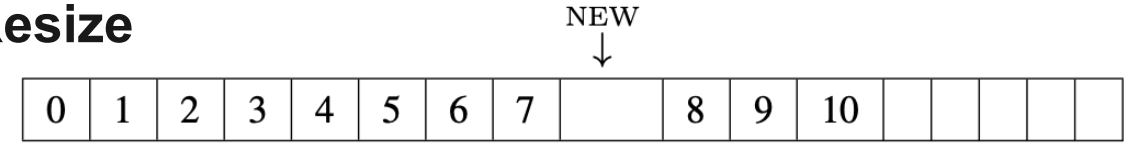
💣 add() / add(mid) / remove(0) / add-/removeFirst()

Aktionen, durch die Elemente bewegt werden (müssen):

- `add()` => manchmal Resize + Kopieren
  => amortisiert `O(1)`



- `add(mid)` => immer Kopieren + manchmal Resize
  => `O(n)`



- `remove(0)` => immer Kopieren
  => `O(n)`

```
[A][B][C][D][E]
 ↓
remove(0)

[B][C][D][E][ ]
 ↑  ↑  ↑  ↑
alles bewegen
```



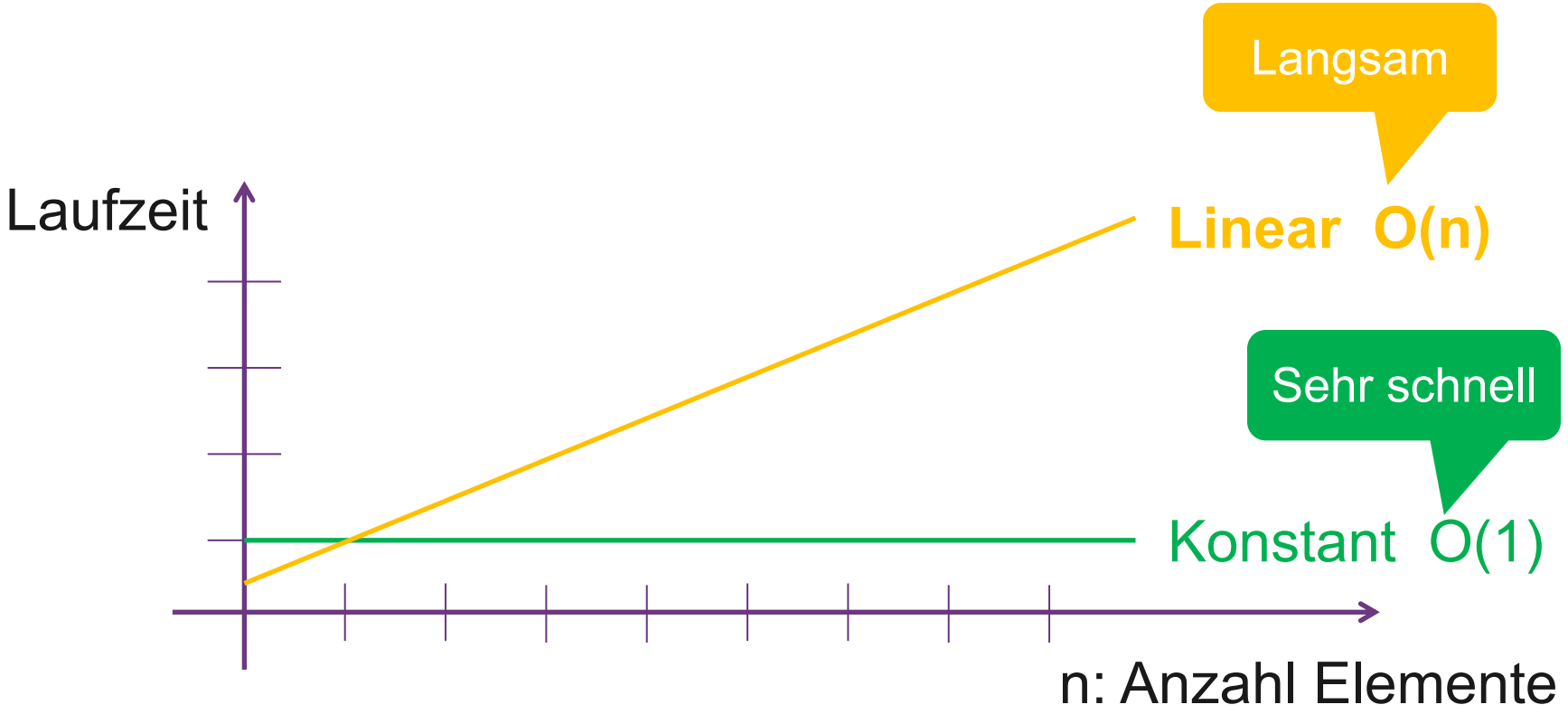
`ArrayList`: Amortisierte Kosten

Average Case

Operation	Methode	Effizienz
Index-Zugriff	`get(), set()`	**Sehr schnell (direkter Zugriff)**
Einfügen eher am Ende	`add()`	**Sehr schnell (amortisiert)**
Einfügen vorne oder Mitte	`add()`	**Langsam (umkopieren)**
Entfernen vorne bis Mitte	`remove(int)`	**Langsam (umkopieren)**
Suchen	`contains()`	**Langsam (durchsuchen)**



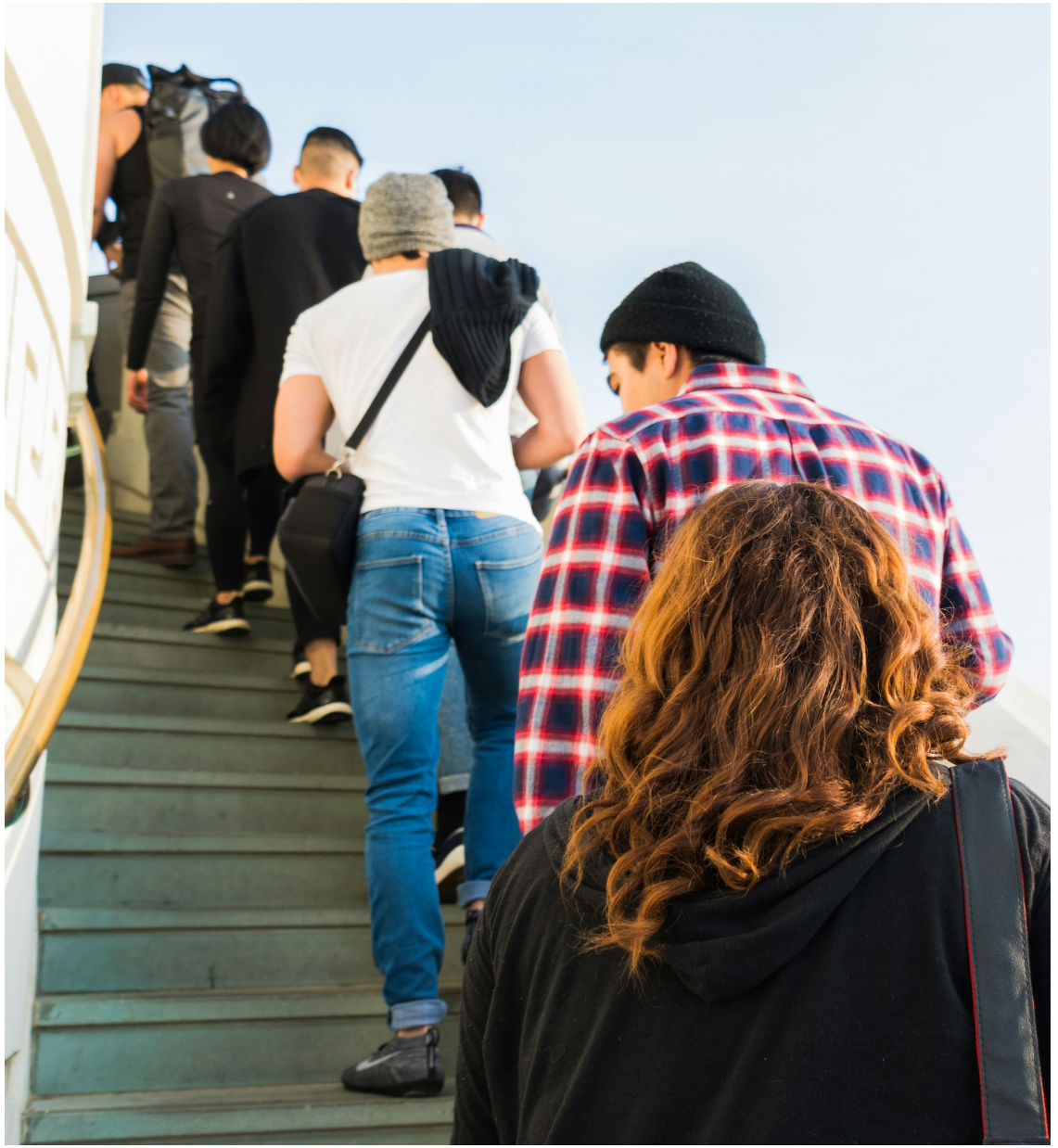
Asymptotische Laufzeitkosten: Linear vs konstant





Queues

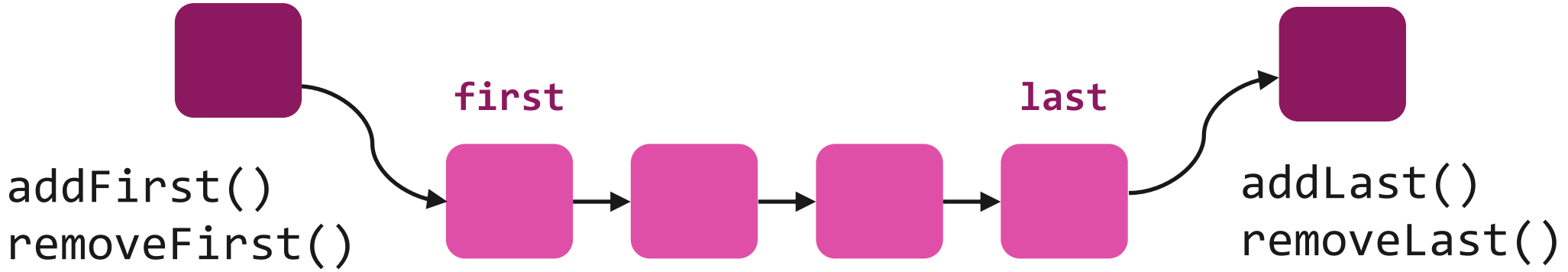
Warteschlangen



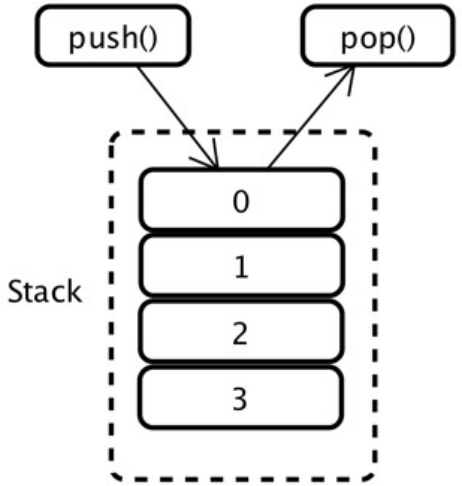


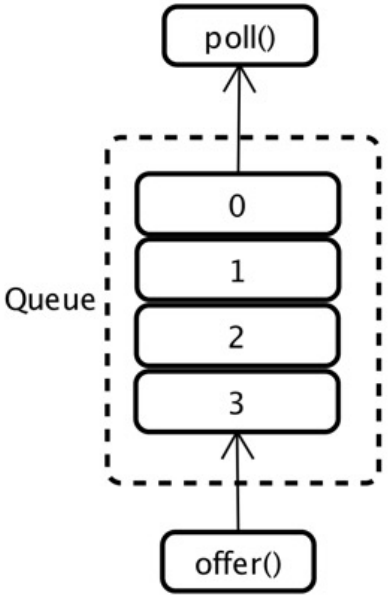
`LinkedList` ist auch eine `Queue`

IDE: FirstQueueExample



Stack
Schreibtischablage
LIFO – Last In First Out





Queue
Schlange an der Kasse
FIFO – First In First Out
FCFS – First Come First Serve



HashSet und HashMap

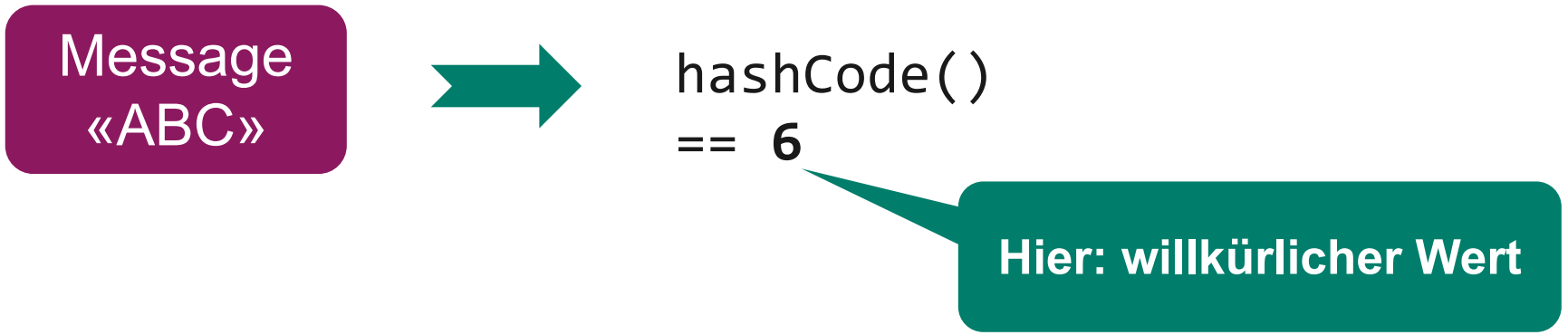
Hashing





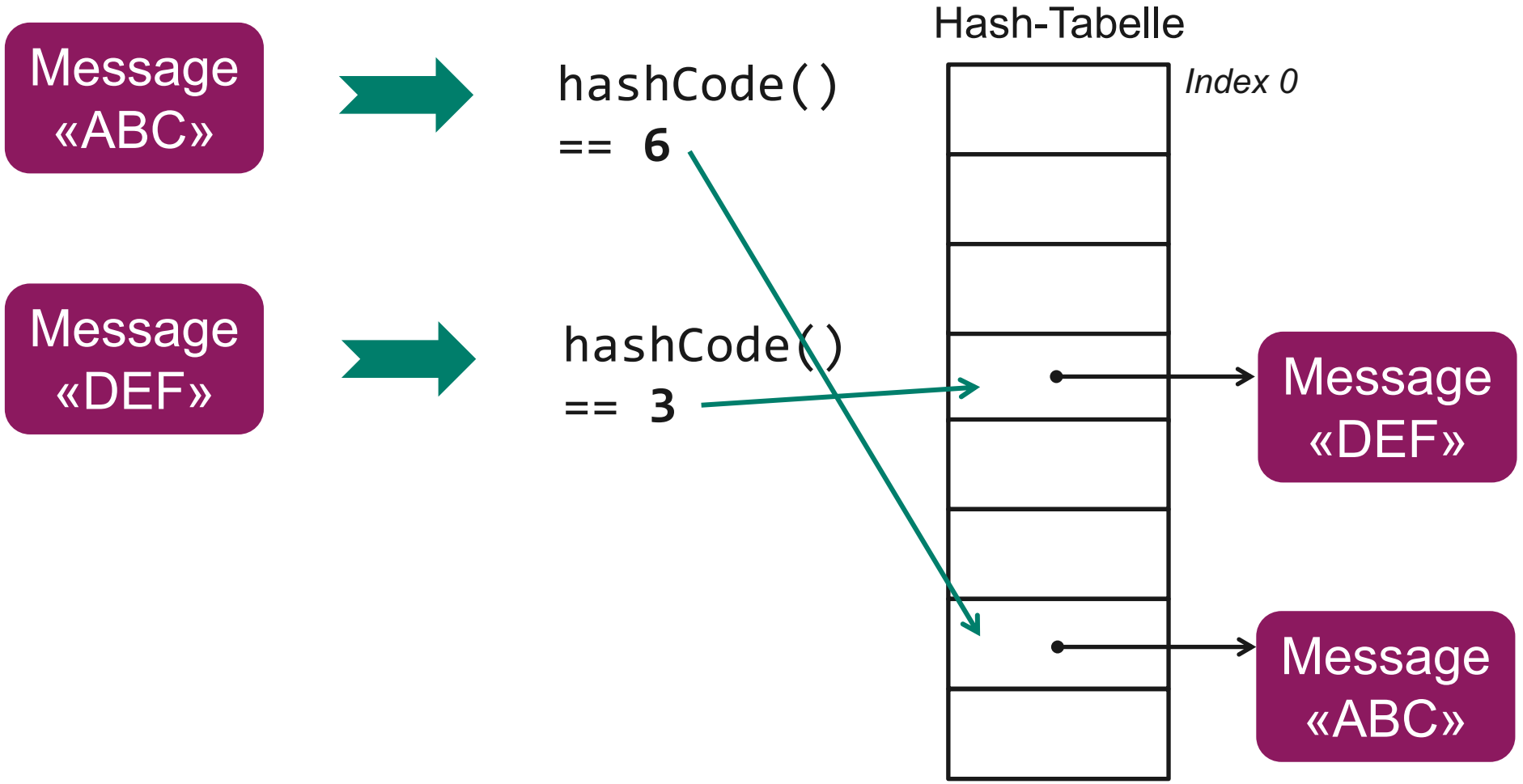
Allgemeines Vorgehen beim Hashing

- Streuspeicher
  - Elemente werden möglichst zufällig/gleichmässig auf ein Array (Hash-Tabelle) verstreut / verteilt
- Hashfunktion
  - Bildet Objekt auf Hash-Code ab
- Rückgabe von `hashCode()`
  - Sagt, wo das Objekt in der Hash-Tabelle hingehört





Hashing: Beispiel

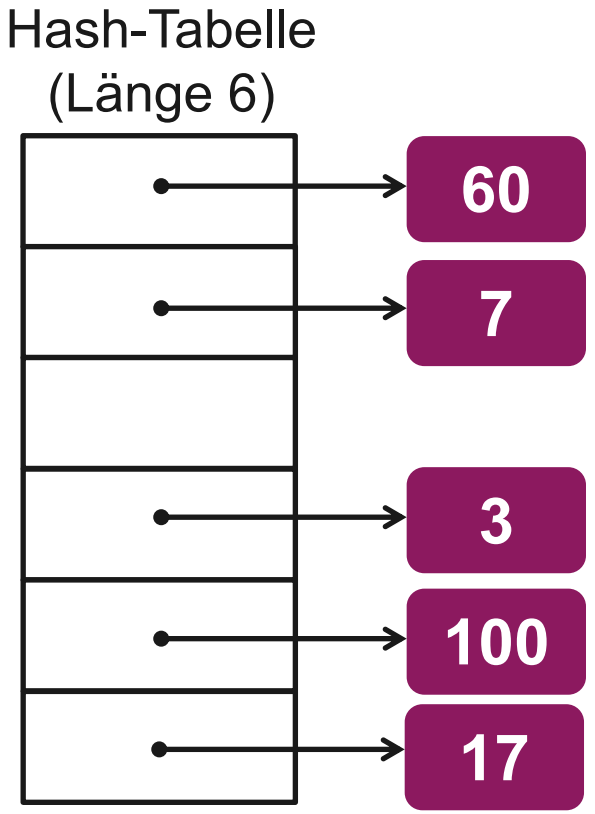




Beispiel: Einfügen

```
var set = new HashSet<Integer>();
```

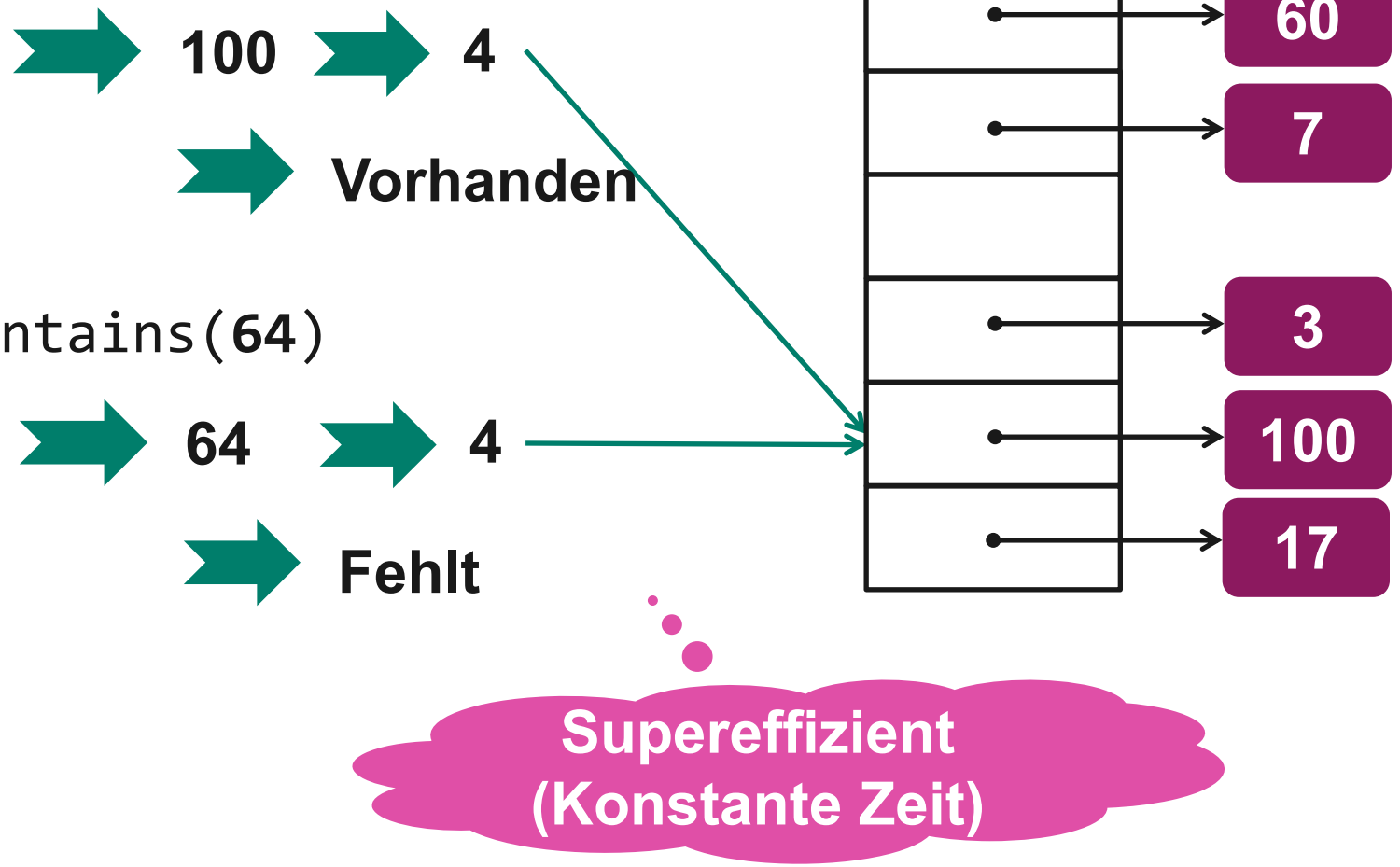
	Hash-Code	*Index*
`set.add(7);`	7	1
`set.add(100);`	100	4
`set.add(3);`	3	3
`set.add(60);`	60	0
`set.add(17);`	17	5





Beispiel: Suchen

```
set.contains(100)
```





Performance-Übersicht

Average Case

	Suchen	Einfügen	Löschen
`ArrayList`	Langsam	Sehr schnell am Ende, langsam sonst	Sehr schnell am Ende, langsam sonst
`LinkedList`	Langsam	Sehr schnell an Enden, langsam sonst	Sehr schnell an Enden, langsam sonst
`HashSet`	Sehr schnell	Sehr schnell	Sehr schnell
`HashMap`	Sehr schnell	Sehr schnell	Sehr schnell
`TreeSet`	Schnell	Schnell	Schnell
`TreeMap`	Schnell	Schnell	Schnell





Was kann man mit Maps sonst noch so machen?



Lookup-Map: Römische Zahlen Buchstaben auf Wert

- Wert ergibt sich normalerweise aus der Addition der Werte der einzelnen Ziffern von links nach rechts beispielsweise XVI für den Wert 16.

```
var valueMap = Map.of('I', 1, 'V', 5, 'X', 10, 'L', 50,
                      'C', 100, 'D', 500, 'M', 1000);
```

1. **Additionsregel**: Gleiche Ziffern nebeneinander werden addiert, etwa XXX = 30 Ebenso gilt dies für kleinere Ziffern nach größeren, z. B. XII = 12.
2. **Wiederholungsregel**: Es dürfen maximal drei gleiche Ziffern aufeinanderfolgen. Nach Regel 1 könnte man die Zahl 4 als IIII schreiben, was diese Regel 2 verbietet. Hier kommt die Subtraktionsregel ins Spiel.
3. **Subtraktionsregel**: Steht ein kleineres Zahlzeichen vor einem größeren, so wird der entsprechende Wert subtrahiert. Schauen wir nochmals auf die 4: Diese kann man als Subtraktion $5 - 1$ realisieren. Das wird im römischen Zahlensystem als IV notiert. Für die Subtraktion gelten folgende Regeln:
   - I steht nur vor V und X
   - X steht nur vor L und C
   - C steht nur vor D und M



Lookup-Map: Römische Zahlen

IDE: RomanNumbersExample

```
public static int fromRomanNumber(final String romanNumber) {
    int value = 0;
    int lastDigitValue = 0;

    for (int i = romanNumber.length() - 1; i >= 0; i--) {
        final char romanDigit = romanNumber.charAt(i);
        final int digitValue = valueMap.getOrDefault(romanDigit, 0);

        final boolean addMode = digitValue >= lastDigitValue;
        if (addMode) {
            value += digitValue;
            lastDigitValue = digitValue;
        }
        else {
            value -= digitValue;
        }
    }
    return value;
}
```




Membership Test





War Story: The Junior Removed Just One Line …

```
public void checkMembership(Collection<String> values, Collection<String> candidates) {
    for (String candidate : candidates) {
        if (values.contains(candidate)) {
            performAction(candidate);
        }
    }
}
```

- **Originalverwendung**:

```
var values = new HashSet<>(origValues)
checkMembership(values, candidates)
```

- **Junior-«Optimierung» «unötiger» Code entfernt:**

```
// var values = new HashSet<>(origValues)
checkMembership(origValues, candidates)
```

`Set` → **O(n)**

`List` → **O(n × m)**

Derselbe Algorithmus
Dieselbe Methode
Dieselben Daten
Eine Zeile entfernt!



Recap: Enthaltensein-Prüfung

- **Enthaltensein-Prüfung mit «`contains()`»**

```
if (myArrayList.contains(value))
```



vs.

```
if (myHashSet.contains(value))
if (myHashMap.contains(key))
```

hash(value)
↓
bucket

Takeaway

```
list.contains() // O(n)
set.contains()  // O(1)*
```




DEMO: `MembershipBenchmark.java`





Recap: Erkenntnis: Auf die Datenstruktur kommt es an

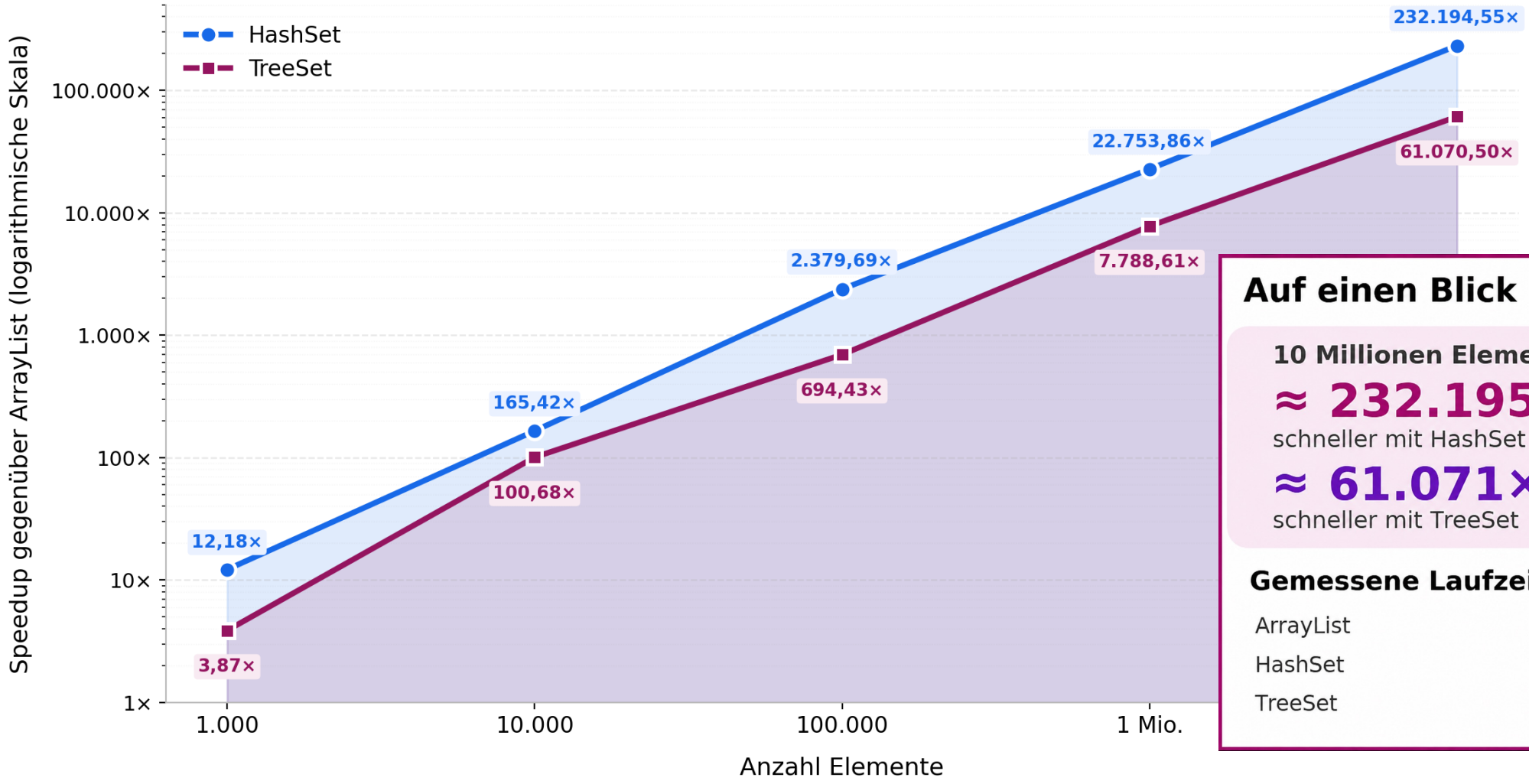
- **Das Programm liefert in etwa eine Tabelle mit Messwerten vergleichbar zu diesen:**

Elemente	ArrayList (ms)	HashSet (ms)	TreeSet (ms)	HS Speedup	TS Speedup
1.000	0,695	0,057	0,179	12,18x	3,87x
10.000	7,320	0,044	0,073	165,42x	100,68x
100.000	87,354	0,037	0,126	2379,69x	694,43x
1.000.000	796,385	0,035	0,102	22753,86x	7788,61x
10.000.000	9046,067	0,039	0,148	232194,55x	61070,50x

- Die **Datenstruktur entscheidet**: `HashSet`/`TreeSet` statt verschachtelter Schleifen – von $O(n^2)$ zu $O(n)$ oder $O(\log n)$.
- **Wachstum** macht den **Unterschied**: Je grösser die Datenmenge, desto stärker wirkt sich die bessere Komplexität aus.
- **Zeit** gegen **Speicher**: Etwas zusätzlicher Speicher ($O(n)$) kann enorm viel Rechenzeit sparen.



Recap: Der Abstand wächst mit jeder Größenordnung!



Auf einen Blick

10 Millionen Elemente

≈ 232.195×
schneller mit HashSet

≈ 61.071×
schneller mit TreeSet

Gemessene Laufzeit

ArrayList	9.046,067 ms
HashSet	0,039 ms
TreeSet	0,148 ms

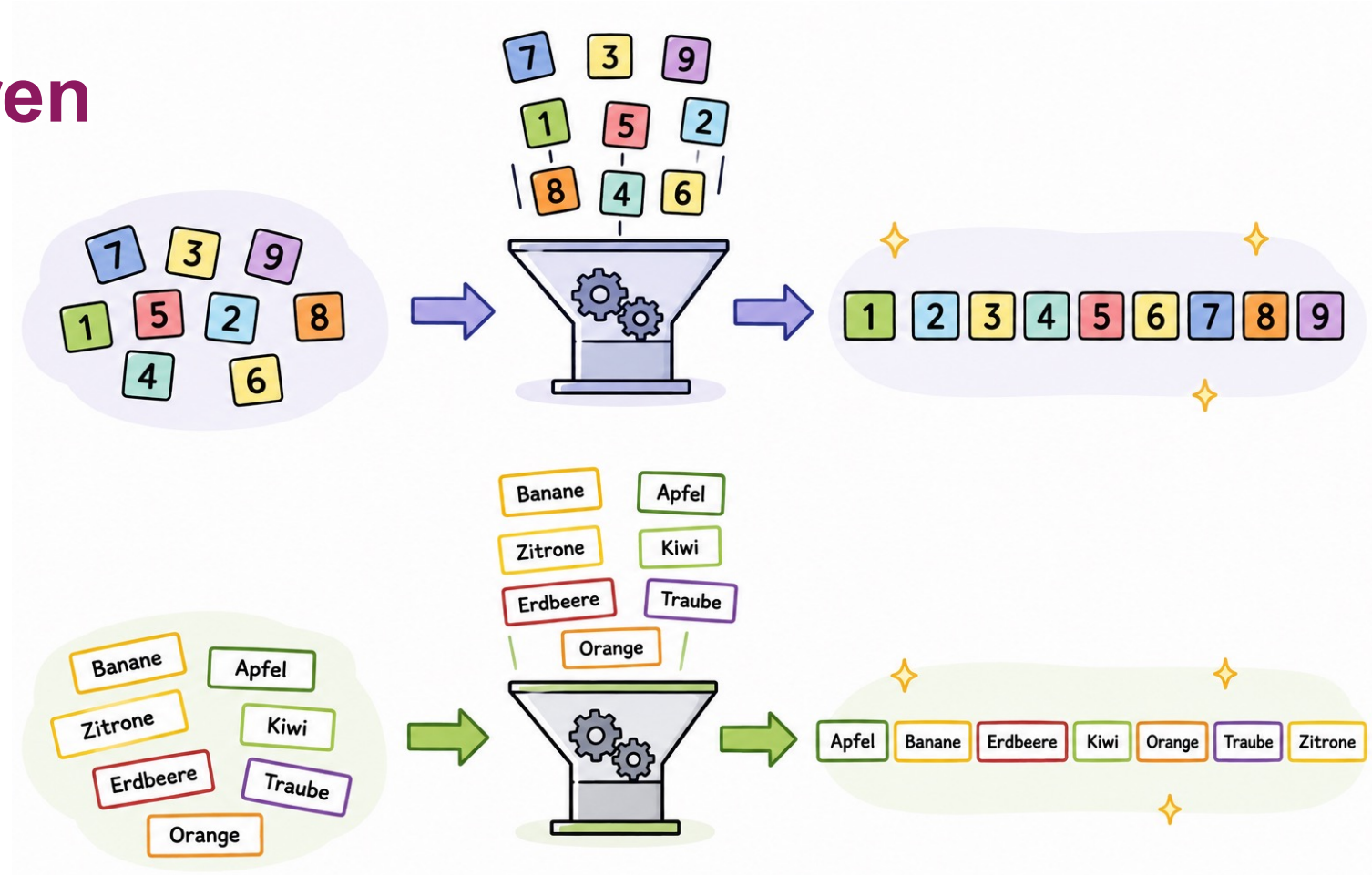


Exercises





🔎 Suchen und Sortieren





Lineare Suche => Binärsuche

O(n) => O(log n)





Recap: Element in einer Liste suchen

- Im einfachsten Fall kann man eine Datenstruktur von vorne nach hinten durchsuchen.
  Das nennt sich lineare Suche:

```
int linearSearch(List<Long> values, long target) {
    for (int i = 0; i < values.size(); i++) {
        if (values.get(i).equals(target)) {
            return i;
        }
    }
    return -1;
}
```



- Diese Suche ist einfach und für kleine Datenbestände absolut in Ordnung. Sie besitzt eine lineare Laufzeit (O(n)). Wird n jedoch sehr groß, so nimmt die Laufzeit deutlich zu.



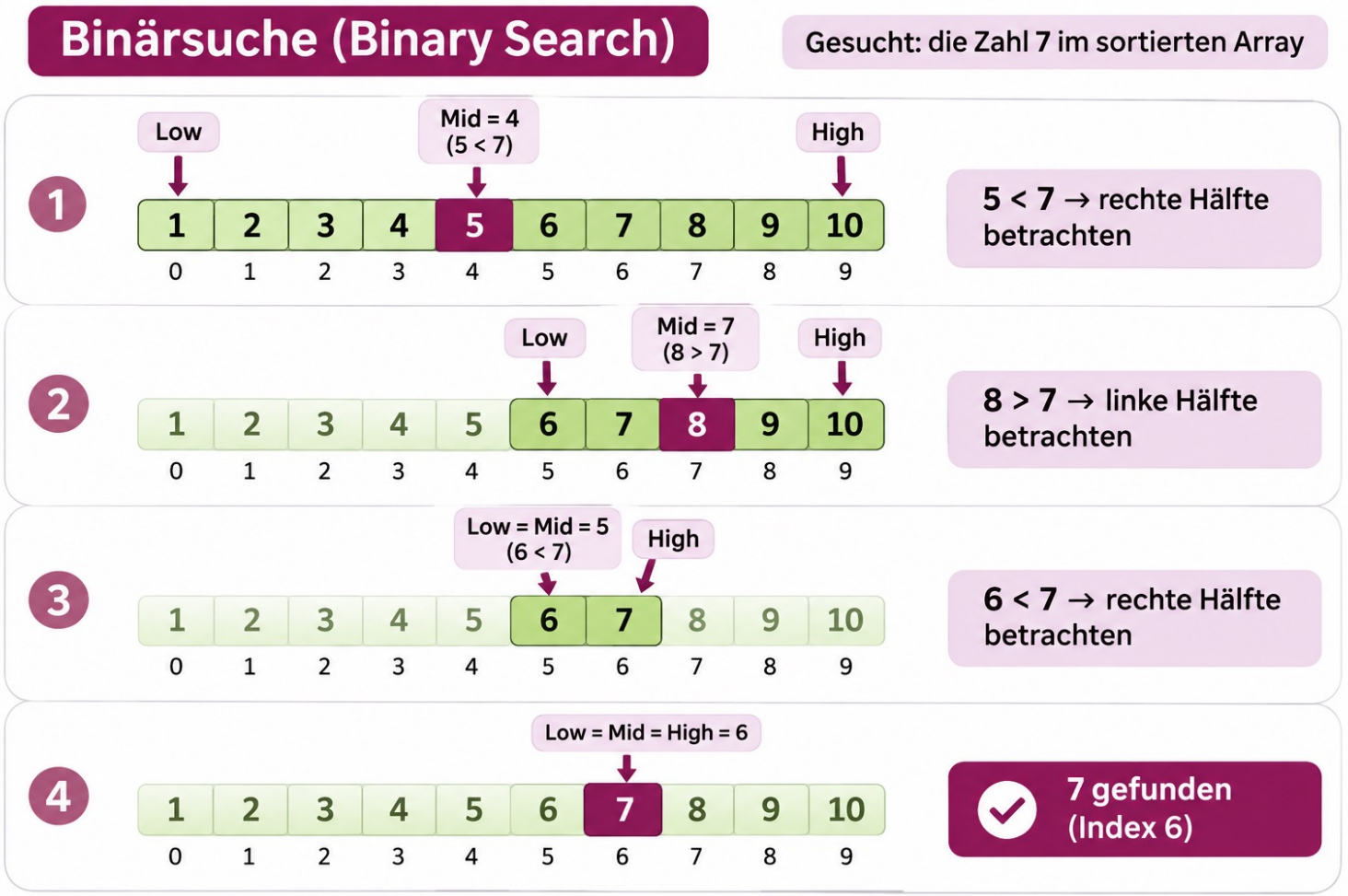


Wie kann man es besser machen?

Was wäre, wenn wir wüssten, dass die Daten *sortiert* vorliegen?



Binärsuche in sortierten Datenbeständen



- Wenn **Daten sortiert**, dann optimierte Suche => **Binärsuche**.
- Sie arbeitet ähnlich, wie man früher in Telefonbüchern gesucht hat.
- Der **Datenbestand** wird in **zwei Hälften** geteilt:
  - Wert in der Mitte anschauen:
    - Ist der Wert gleich, ist die Suche beendet.
    - Ist der zu suchende **Wert kleiner oder grösser** als der Wert in der Mitte, dann in der **unteren** bzw. **oberen Hälfte** weitersuchen



Binärsuche rekursiv

- Das Vorgehen kann man **rekursiv** recht einfach implementieren, indem der zu durchsuchende **Datenbestand** immer auf die **passende Hälfte verkleinert** wird, bis er entweder leer ist oder aus einem Element besteht.

```
static int binarySearchRecV1(List<Long> values, int target) {
    if (values.isEmpty())
        return -1;

    int mid = (values.size() - 1) / 2;
    if (values.get(mid) == target)
        return mid;
    else if (values.get(mid) < target)
        return binarySearchRecV1(values.subList(mid + 1, values.size()), target);
    else
        return binarySearchRecV1(values.subList(0, mid), target);
}
```

O(1) – `subList()` erzeugt anders als Slicing in Python keine Kopie, sondern einen View





Was ist daran trotzdem noch ungünstig?

In der Realisierung entspricht `mid` immer der Position im kleinsten Abschnitt und nicht dem Wert der Position des Originaldatenbestands*



Binärsuche rekursiv

- Mit ein wenig Nachdenken lässt sich durch die Two-Pointer-Technik das Erzeugen von Sublisten vermeiden und der Fehler in der Indexberechnung ebenfalls beheben:

```
static int binarySearchRecV2(List<Long> values, int target) {
    return binarySearchRecV2Helper(values, target, 0, values.size() - 1);
}

static int binarySearchRecV2Helper(List<Long> values, int target, int left, int right) {
    if (left > right) return -1;

    int mid = (left + right) / 2;
    if (values.get(mid) == target)
        return mid;
    else if (values.get(mid) < target)
        return binarySearchRecV2Helper(values, target, mid + 1, right);
    else // if (values.get(mid) > target)
        return binarySearchRecV2Helper(values, target, left, mid - 1);
}
```




Binärsuche iterativ

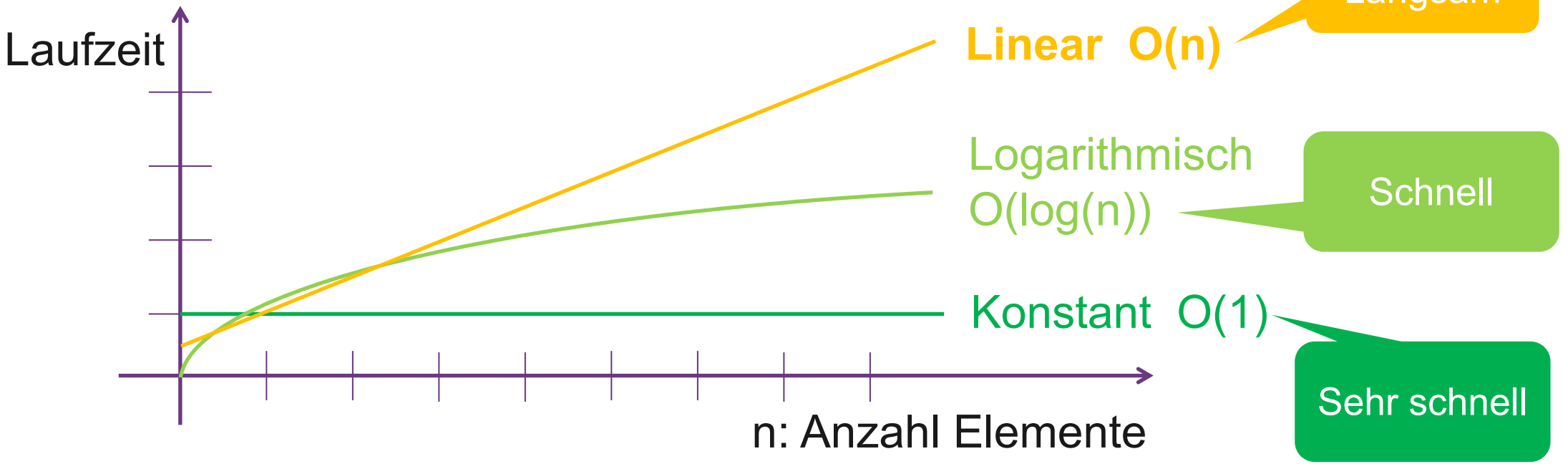
- Mit Two Pointer kann man die Binärsuche **iterativ** wie folgt implementieren:

```
static int binarySearchIter(List<Long> values, int target) {
    int left = 0;
    int right = values.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (values.get(mid) == target)
            return mid;
        else if (values.get(mid) < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```




O(n) => O(log n) klingt zunächst unspektakulär …



Größe des Datenbestands (n)	$O(n)$	$O(\log_2 n)$	$O(1)$
10	10	~ 4	1
1.000	1.000	~ 10	1
1.000.000	1.000.000	~ 20	1
1.000.000.000	1.000.000.000	~ 30	1



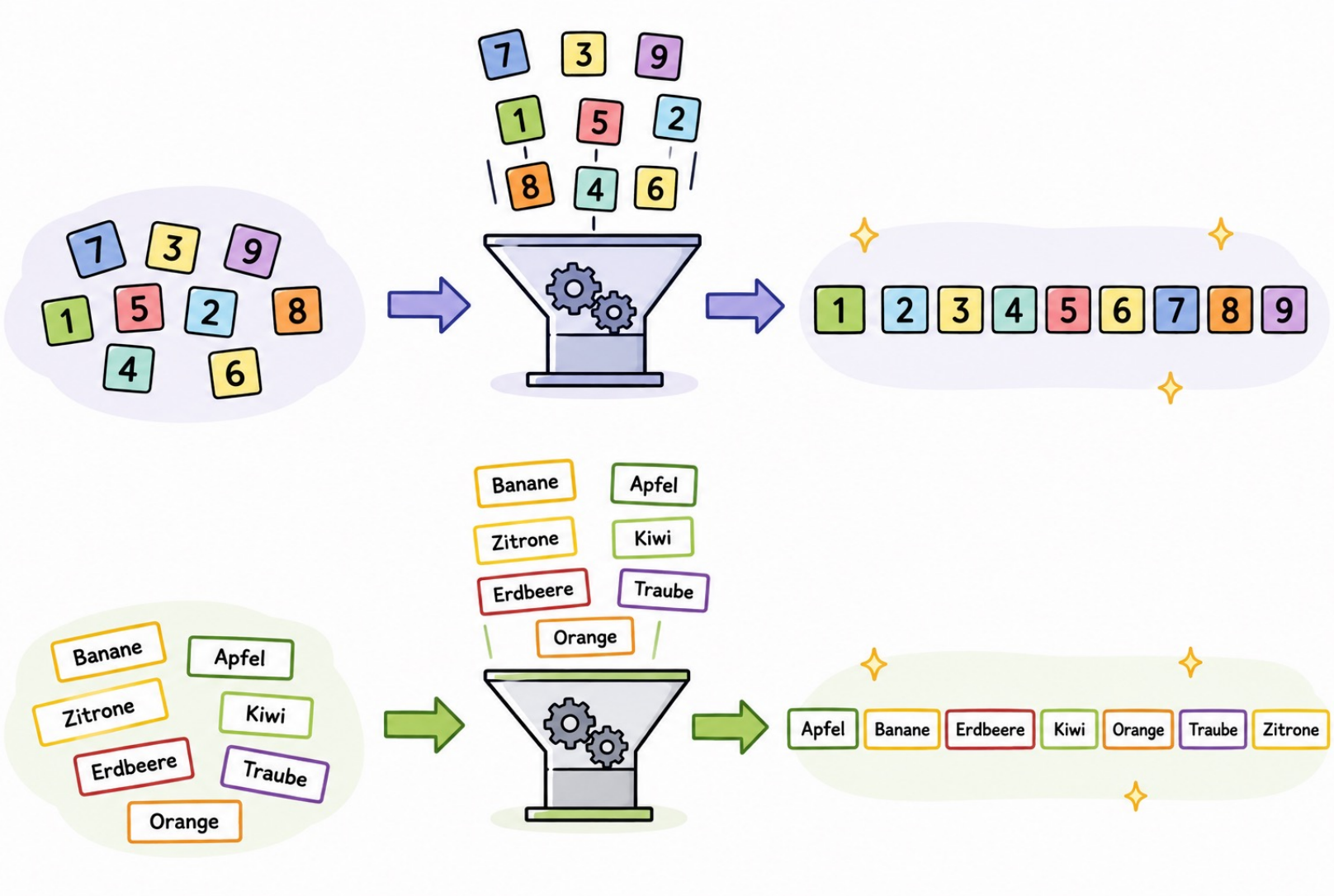
Sortieren

Insertion Sort

Selection Sort

Merge Sort

Quick Sort





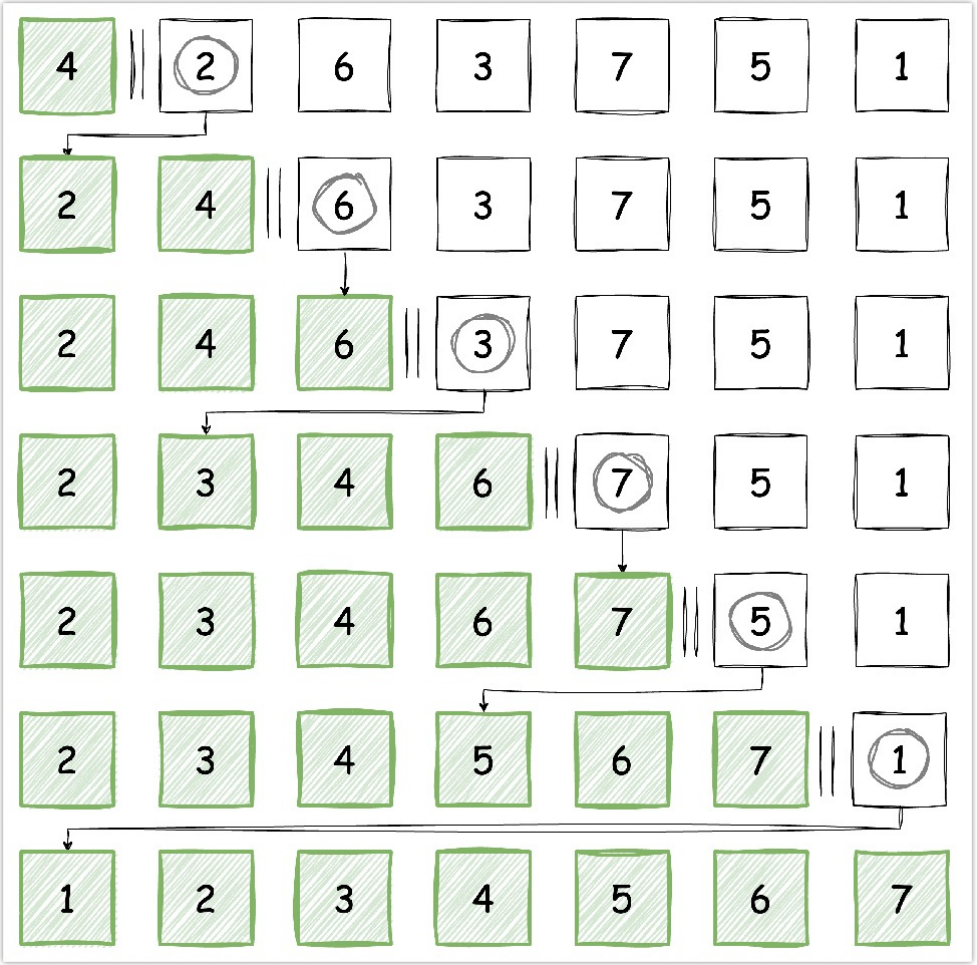


„Wie sortierst du Spielkarten?“



Insertion Sort – «Spielkarten sortieren»

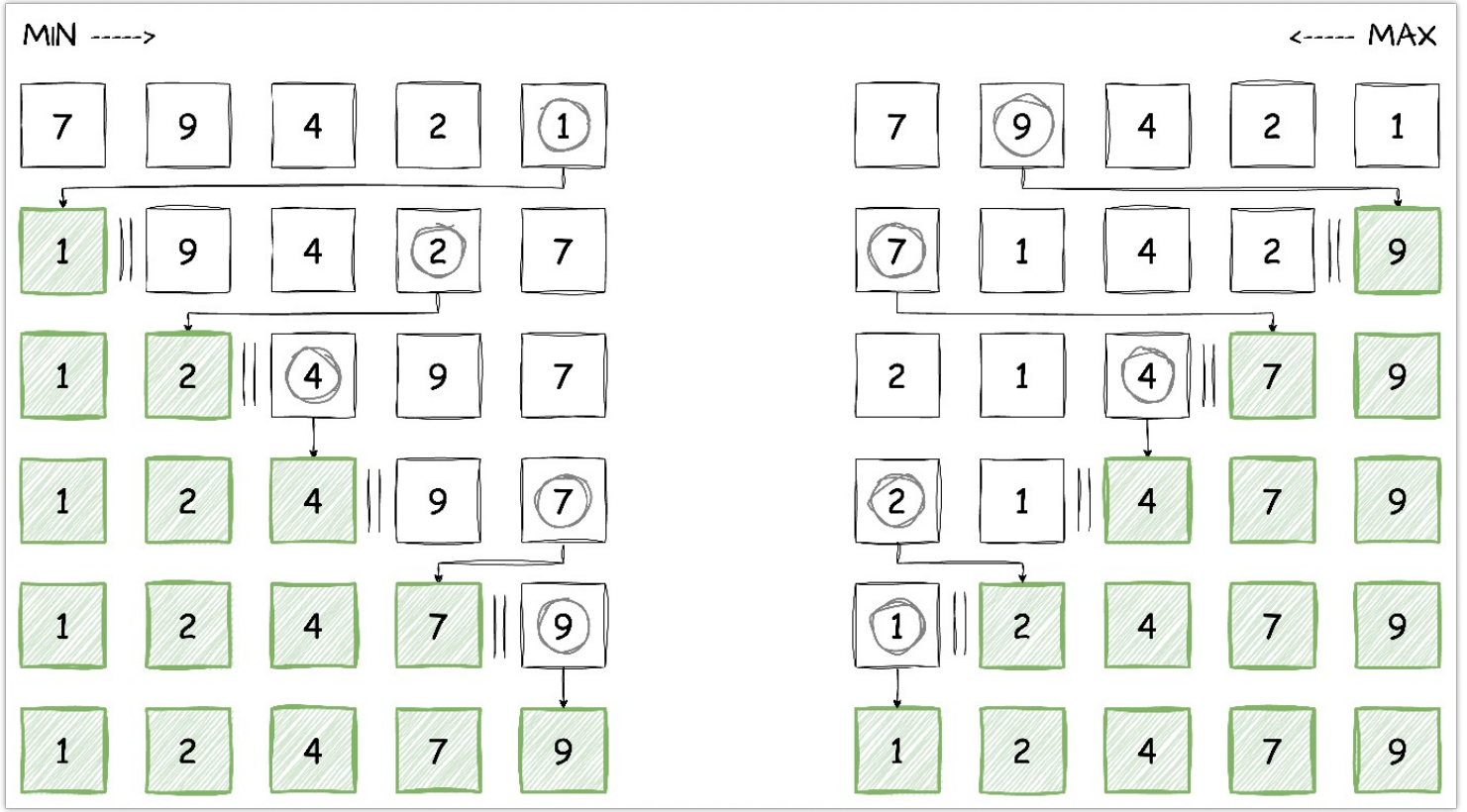
- Insertionsort wie Sortieren von Karten in der Hand
- **Man baut schrittweise einen sortierten Bereich auf.**
- Jedes neue Element wird an die passende Position eingefügt.
- **Vorgehen**: Man beginnt auf der linken Seite, nimmt dann immer die nächste rechts benachbarte Karte und fügt diese passend in den bereits sortierten linken Teil ein, wodurch in der Regel einige der bereits sortierten Karten nach rechts wandern.
- Besonders gut bei kleinen oder bereits weitgehend sortierten Eingaben.
- Typisch: $O(n^2)$, Best Case $O(n)$.





Selection Sort – «Suche immer das Minimum/Maximum»

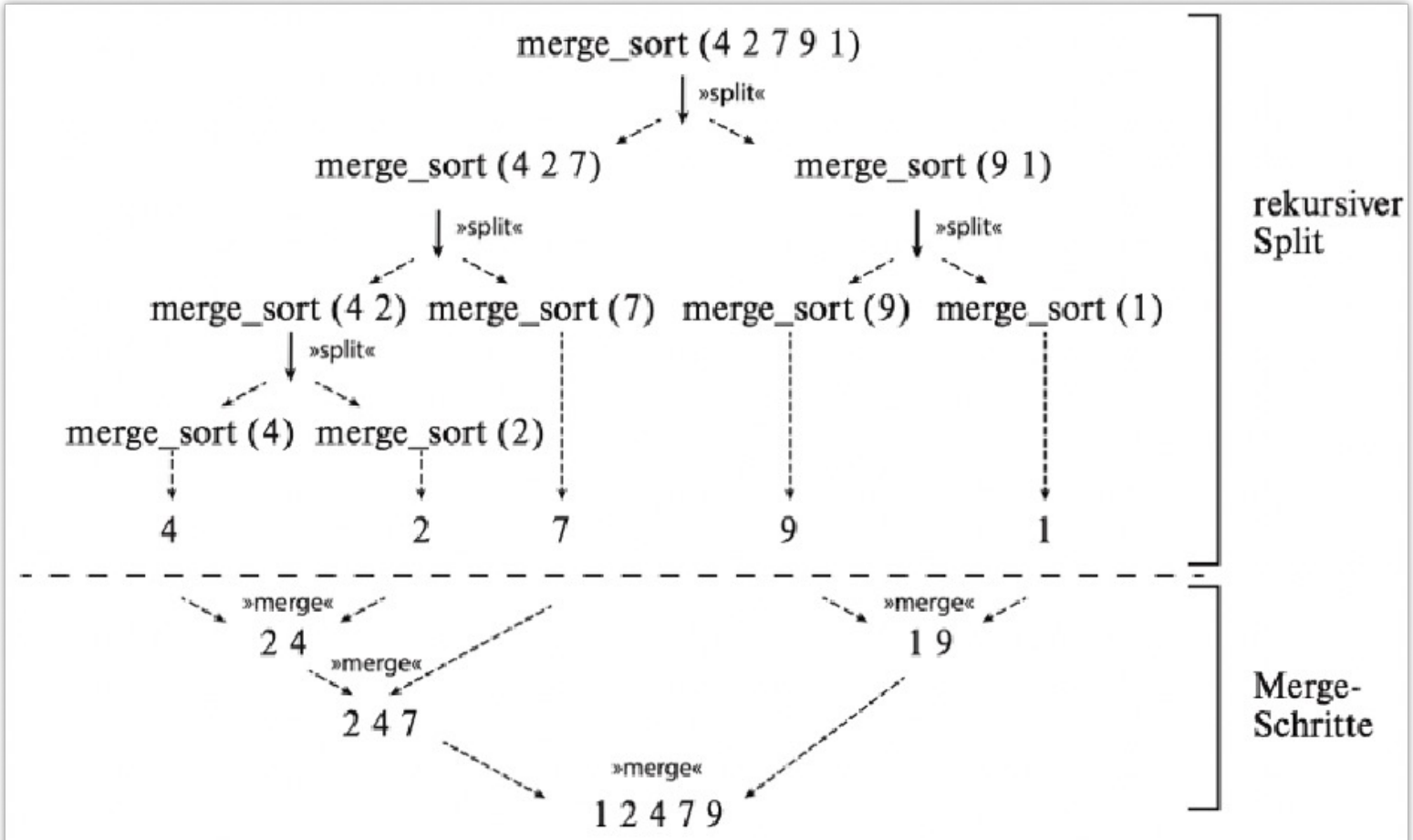
- Sucht im unsortierten Teil das kleinste / größte Element.
- Tauscht es an die nächste freie Position.
- Einfach zu verstehen, aber die Anzahl der Vergleiche ist hoch.
- Typisch: $O(n^2)$, auch im Best Case!





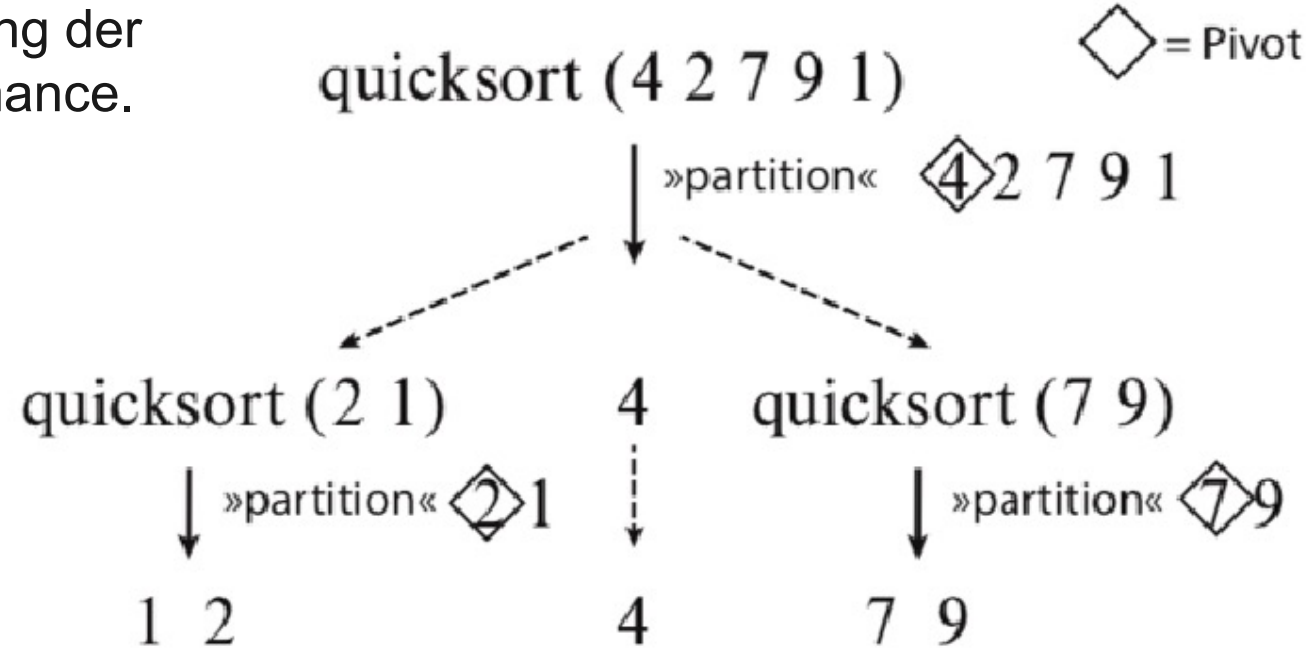
Merge Sort – «Teile und Herrsche / Split und Merge»

- Mergesort basiert auf einem »Divide-and-Conquer-Ansatz«
- **Teilt** die Eingabe in kleinere **Datenbereiche** bis nur noch kein oder 1 Element.
- Führt dann die Teilbereiche Schrittweise sortiert zusammen.
- **Garantierte Laufzeit: $O(n \log n)$**
- Benötigt zusätzlichen Speicher für das Mischen (die Zwischen-schritte).



Quick Sort – «Teile und Herrsche / Partition and Combine»

- Auch «Divide-and-Conquer-Ansatz» und zerteilt die zu sortierende Liste in immer kleinere Teile.
- Wählt ein Pivot und partitioniert die Elemente gemäß kleiner als oder größer gleich Pivot-Wert.
- Sortiert die entstehenden Teilbereiche rekursiv.
- Durchschnittlich $O(n \log n)$, im Worst Case $O(n^2)$.
- Die Pivot-Wahl und die Verteilung der Daten beeinflussen die Performance.





Gleiche Aufgabe, aber stark abweichendes Wachstum

Algorithmus	Best Case	Durchschnitt	Worst Case
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$*	$O(n \log n)$	$O(n^2)$

- $O(n \log n)$ ist keine Garantie dafür, dass Merge Sort in jedem konkreten Fall schneller ist als Insertion Sort.
- Bei kleinen Datenbeständen kann ein geringerer Verwaltungsaufwand eines einfachen Verfahrens durchaus vorteilhaft sein.
- Big O ist ein Modell für Skalierung – kein universelles Ranking für jede Eingabegröße.





„Mit welcher zunächst absurd klingenden Idee kann man Quick Sort auf nahezu garantiert O(n log n) bringen?“



DEMO: `Sortier Arena`





Recap: Sortierverfahren

- **Insertion Sort**
  Es wird immer das nächste Element genommen und an die richtige Stelle im bereits sortierten linken Teil einsortiert.
- **Selection Sort**
  Das aktuell kleinste/größte Element im unsortierten Bereich wird markiert und nach vorne getauscht. Es sind viele Vergleiche trotz weniger Tauschoperationen nötig sind.
- **Merge Sort**
  Die Folge wird in Teilbereiche zerlegt, anschliessend werden die sortierten Bereiche wieder zusammengeführt. Hier sind eine Baumansicht der Rekursion und die Balkenanimation besonders lehrreich.
- **Quick Sort**
  Ein Pivot teilt den Datenbestand in zwei Bereiche kleiner als und größer oder gleich. Die Wahl des Pivot-Elements beeinflusst Vergleiche und Größe der Partitionen.



Beim Sortieren geht’s nicht bloß um Sortieren ☺

- **Sortieren kann sich lohnen, wenn es nachfolgende Operationen günstiger macht.**
- **Beispiel: 1.000 Suchanfragen auf denselben Daten**
  - Unsortierte Liste: jede lineare Suche kostet im Worst Case O(n).
  - Einmal sortieren: O(n log n).
  - Danach binär suchen pro Suche: O(log n).
- **Gesamtkosten für q Suchen vereinfacht**:
  - Linear suchen: O(q n)
  - Sortieren und binär suchen: O(n log n + q log n)

Sobald q grösser als ungefähr log n ist, ist O(n log n + q log n) asymptotisch günstiger als O(q n).

- **Kernaussage**: Eine zusätzliche Arbeit am Anfang kann die Gesamtlösung beschleunigen, wenn sich die Daten anschließend oft nutzen lassen.



Exercises





💡 Cleverere Lösungen: Two Pointers und HashMaps





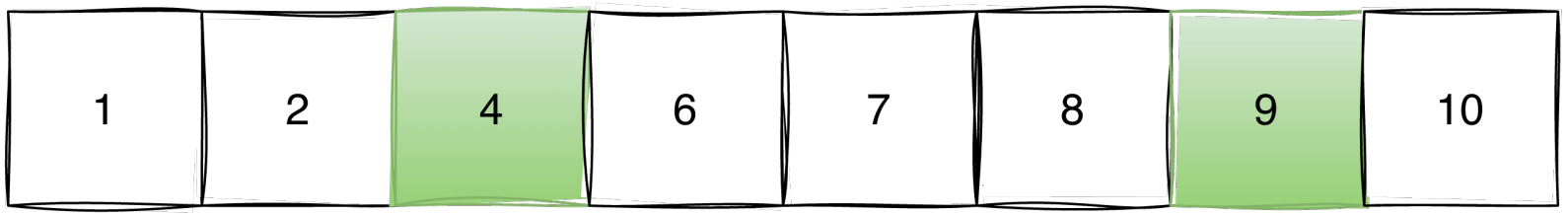
Two Pointers





Pair Sum Problemstellung

- Es sei ein duplikatfreies Array bzw. eine ebensolche Liste von Ganzzahlwerten gegeben, die aufsteigend sortiert sind.
- Die Aufgabe besteht nun darin, herauszufinden, ob es ein **Pärchen der Zahlen** gibt, dessen **Summe den Wert k** ergeben.



Summe: 13

- Ergebnis als Reord modellieren:

```
record IndexPair(int left, int right) {}
```




Pair Sum Brute-Force-Implementierung und Erkenntnis

```
static IndexPair pairSumOrderedBruteForce(List<Integer> values, int k) {
    for (int i = 0; i < values.size(); i++) {
        for (int j = i + 1; j < values.size(); j++) {
            if (values.get(i) + values.get(j) == k)
                return new IndexPair(i, j);
        }
    }
    return new IndexPair(-1, -1);
}
```

- Diese Art von Brute-Force-Umsetzung mit zwei ineinander verschachtelten Schleifen sieht man häufiger zur Lösung verschiedener Problemstellungen.
- Gut verständlich, aber Laufzeitkomplexität von $O(n^2)$. Für kleine Datenbestände oft noch okay.
- **Mit steigender Anzahl an Eingabewerten wird der Effekt deutlich negativ spürbar und es kann zu störend längen Ausführungszeiten kommen.**



DEMO: `PairSumBruteForce.java`





Idee: Positionszeiger

- Daten eines Arrays oder einer Liste werden gewöhnlich indiziert zugegriffen, etwa um einen Wert auszulesen. Dazu benötigt es einen Positionszeiger («Pointer») bzw. Index.
- Wenn man jedoch einen weiteren Positionszeiger einführt, erweitert man direkt die Möglichkeiten: Vergleiche oder Berechnungen möglich.
- Es gibt verschiedene Ausprägungen von Two Pointers in der Art und Weise, wie und in welche Richtung die Positionszeiger bewegt werden.
- Ein hilfreiche Variante ist, die beiden Pointer von ganz links und ganz rechts nach innen auf sich zu bewegen. Diese Schritte können auch an Bedingungen geknüpft werden.

```
[ … , … , …., mid , …, … , … ]
    left =>          <= right
```





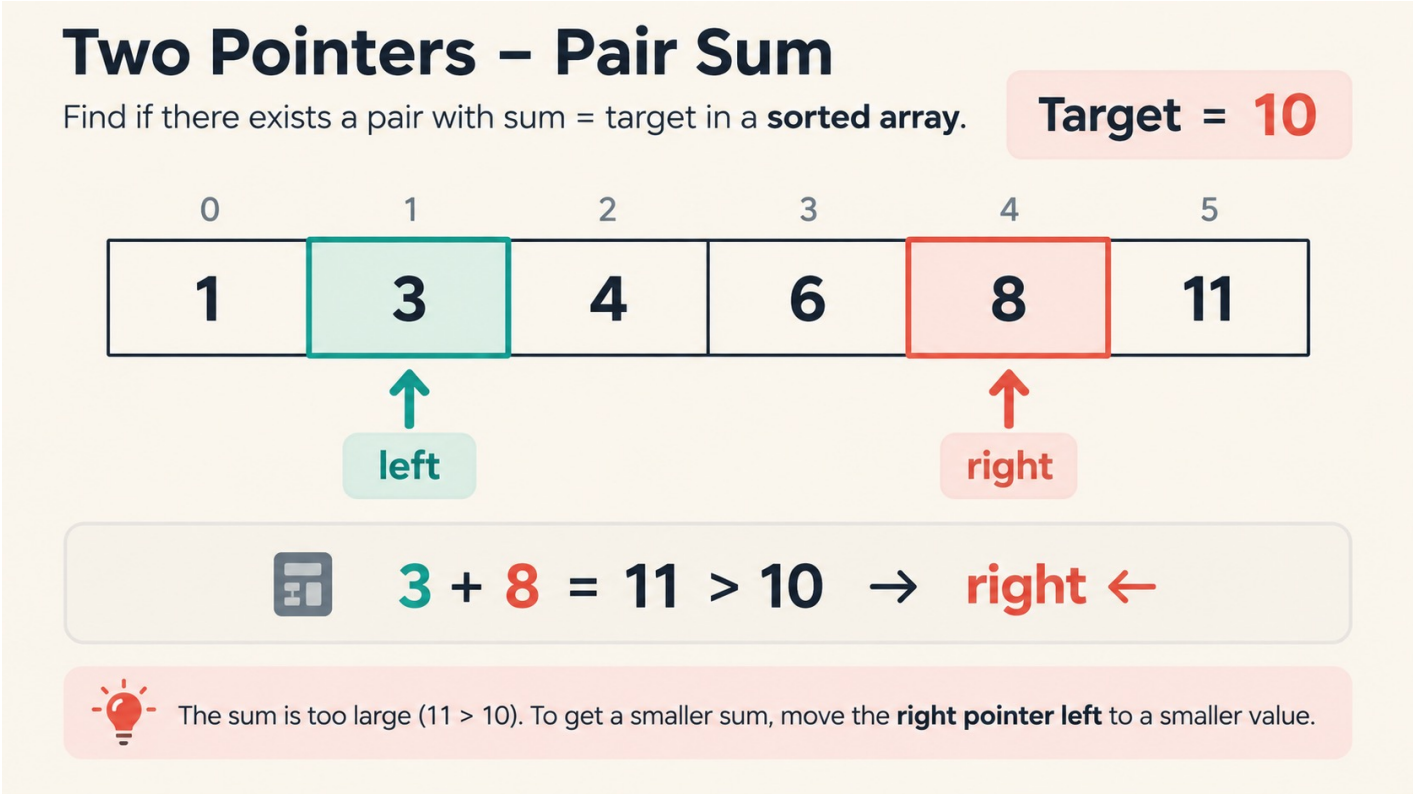

Wie kann man die Brute-Force-Variante mit dem Wissen um Two Pointers und Sortierung besser machen?



Optimierung Pair Sum mit Two Pointers

- Positionszeiger beginnen ganz links und ganz rechts. Dann lässt sich die Summe aus den referenzierten Werten bilden. Summe ungleich erwarteter Wert, dann Positionszeiger bewegen:

  left – Wenn wir links nach rechts bewegen: die Summe wird größer oder bleibt zumindest gleich.
  right – Wenn wir rechts nach links bewegen: die Summe wird kleiner oder bleibt zumindest gleich.





Optimierung Pair Sum mit Two Pointers

```
values = [-5, -1, 2, 7, 11]  k = 1

[-5, -1, 2, 7, 11]
  l              r        sum = -5 + 11 = 6 => right -= 1

[-5, -1, 2, 7, 11]
  l         r             sum = -5 + 7 = 2 => right -= 1

[-5, -1, 2, 7, 11]
  l      r                sum = -5 + 2 = -3 => left += 1

[-5, -1, 2, 7, 11]
      l  r                sum = -1 + 2 = 1 => GEFUNDEN
```




DEMO: `PairSumOrdered_TwoPointer.java`





Anagramme

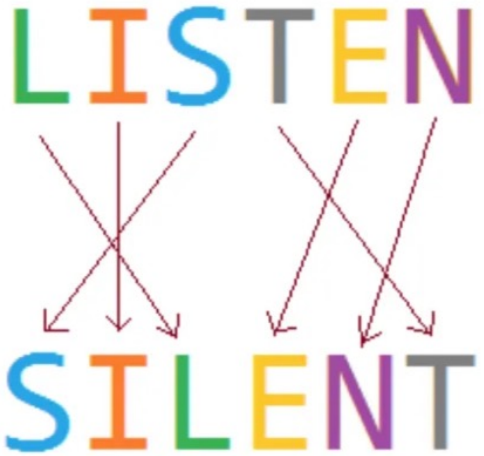
$O(n^2)$ => O(n) / O(n log n)





Anagramme Problemstellung

- Anagramme sind Worte, die die gleichen Buchstaben, aber in unterschiedlicher Reihenfolge enthalten, etwa «Ampel», «Palme» und «Lampe».



- Es gibt spontan die Lösungsidee, etwa alle Kombinationen aufzubereiten und diese dann mit dem zweiten Wort zu vergleichen. Ganz offensichtlich ist das ungünstig bezüglich Laufzeit und Speicherverbrauch.



Anagramme Problemstellung und Brute-Force-Lösung

- Brute-Force-Variante, die zwei verschachtelte Schleifen für beide Wörter benutzt und jeweils durch das Wort läuft und Buchstaben auf Existenz prüft und in dem Fall «wegstreicht»:

```
Wort 1          Wort 2      Wort 2 (Alternative)
Ampel           Lampe       Palme
^                ^           ^
Ampel           Lampe       Palme
 ^                ^           ^
Ampel           Lampe       Palme
  ^                ^          ^
Ampel           Lampe       Palme
   ^                ^          ^
Ampel           Lampe       Palme
    ^                ^          ^
Ampel           Lampe       Palme
     ^                ^          ^          => Anagramm
```



Anagramme Brute-Force-Implementierung

```
static boolean isAnagramBruteForce(String word1, String word2) {
    if (word1.length() != word2.length())
        return false;

    word1 = word1.toLowerCase();
    word2 = word2.toLowerCase();
    var word2_with_strike_through = word2.toCharArray();

    for (int i = 0; i < word1.length(); i++) {
        boolean letter_found = false;
        for (int j = 0; j < word2_with_strike_through.length; j++) {
            if (word1.charAt(i) == word2_with_strike_through[j]) {
                letter_found = true;
                word2_with_strike_through[j] = '-';
                break;
            }
        }
        if (!letter_found)
            return false;
    }
    return true;
}
```






Wie kann man es performater machen?

Wir kennen bereits Two Pointers, Sortierung und `HashMaps`!

Was können wir wie nutzen?



DEMO: `AnagramsWithBenchmark.java`





Erkenntnis: O(n) ist nicht immer schneller als O(n log n)

- **Die asymptotische Komplexität beschreibt das Wachstum – nicht die tatsächliche Laufzeit.**

```
=== 50’000 Zeichen ===
Algorithmus          Median (ms)   Mittel (ms)        Faktor
-------------------------------------------------------------
Brute Force            568.8661      566.0786      2059.55x
HashMap                  0.6392        0.6355         2.31x
Sorting                  0.2762        0.2758         1.00x

=== 100’000 Zeichen ===
Algorithmus          Median (ms)   Mittel (ms)        Faktor
-------------------------------------------------------------
Brute Force           2235.9907     2240.2060      5891.51x
HashMap                  1.2669        1.2689         3.34x
Sorting                  0.3795        0.3823         1.00x
```




Erkenntnis: O(n) ist nicht immer schneller als O(n log n)

- **Die asymptotische Komplexität beschreibt das Wachstum – nicht die tatsächliche Laufzeit.**

```
Algorithmus          Median (ms)   Mittel (ms)        Faktor
=== 50’000 Zeichen ===
HashMap                   0.6392        0.6355         2.31x
Sorting                   0.2762        0.2758         1.00x

=== 100’000 Zeichen ===
HashMap                   1.2669        1.2689         3.34x
Sorting                   0.3795        0.3823         1.00x
```

- Variante Sorting ist bei 50.000 Zeichen etwa 2,31-mal schneller als `HashMap`.
- Variante Sorting ist bei 100.000 Zeichen etwa 3,34-mal schneller als `HashMap`.
- **Obwohl Sorting asymptotisch schlechter abschneidet, ist es in hier praktisch schneller.**





Woran könnte das liegen?



Warum ist O(n) nicht immer schneller als O(n log n)?

- **Die Big-O-Notation berücksichtigt nicht die konkreten Kosten einzelner Operationen.**
- `HashMap`: Hash-Berechnung, Hashing, Buckets, Objektverwaltung und gegebenenfalls Boxing.
- Sorting: Optimierte Sortieralgorithmen, effiziente Speicherzugriffe und geringe Verwaltungsaufwände.
- **Ein O(n)-Algorithmus kann also höhere Konstanten haben als ein O(n log n)-Algorithmus.**

Big O ist nicht alles!

O(n) kann langsamer sein als O(n log n).

Entscheidend ist nicht nur, wie viele Operationen wachsen – sondern auch, wie teuer diese Operationen sind.



Exercises





Weiter zu Teil 2 …



